

Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

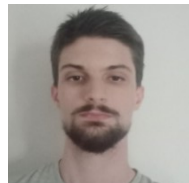
Unidade Curricular de Projeto de Engenharia de Conhecimento

Ano Letivo de 2025/2026

Relatório Técnico-Científico Grupo 4



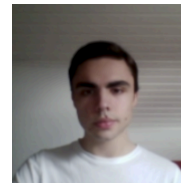
Carlos Fernandes
pg60239



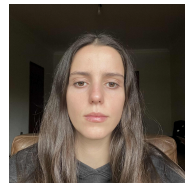
Diogo Outeiro
pg60246



António Cruz
pg61512

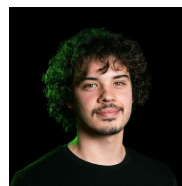


Diogo Cunha
pg60248



**Matilde
Fernandes**
pg57588

Junho, 2026



Miguel Luzes
pg55907

EC

Data da Receção	
Responsável	
Avaliação	
Observações	

Relatório Técnico-Científico

Carlos Fer-
nandes
pg60239

Diogo Ou-
teiro
pg60246

António
Cruz
pg61512

Diogo Cu-
nha
pg60248

Matilde
Fernan-
des
pg57588

Miguel
Luzes
pg55907

Resumo (Abstract)

Este projeto teve como objetivo o desenvolvimento de um sistema inteligente de suporte à decisão focado na área da medicina preventiva e análise de dados em saúde. A crescente quantidade de informação clínica e epidemiológica disponível torna necessária a criação de ferramentas capazes de auxiliar profissionais e utilizadores na interpretação de dados médicos de forma acessível e eficiente.

Como solução, foi desenvolvido um sistema composto por um *chatbot* inteligente integrado com *dashboards* analíticos interativos. O sistema combina modelos de linguagem, mecanismos de *Retrieval Augmented Generation* (RAG), bases de dados heterogêneas e ferramentas especializadas para responder a questões relacionadas com doenças, indicadores de saúde, medicamentos e estatísticas epidemiológicas.

A arquitetura implementada inclui pipelines de ingestão e transformação de dados, bases de dados SQL, MongoDB e vetoriais, bem como um agente capaz de selecionar diferentes ferramentas consoante o tipo de pergunta efetuada pelo utilizador. O sistema suporta também atualização dinâmica de *dashboards* através de linguagem natural.

Durante o desenvolvimento foram integradas múltiplas fontes de dados médicas e epidemiológicas, incluindo datasets com mais de 100 mil registos. O resultado final corresponde a um protótipo funcional capaz de combinar capacidades analíticas, pesquisa semântica e interação conversacional numa única plataforma.

Agradecimentos

Gostaríamos de agradecer ao docente da unidade curricular pelo acompanhamento, orientação e *feedback* contínuo ao longo do desenvolvimento deste projeto.

Agradecemos também a todos os membros da equipa pelo trabalho colaborativo, dedicação e entreaajuda demonstrados durante as diferentes fases do projeto, desde a engenharia de dados até ao desenvolvimento do sistema inteligente e da interface visual.

Por fim, agradecemos às comunidades *open-source* e às ferramentas utilizadas no desenvolvimento da solução, nomeadamente bibliotecas de inteligência artificial, *frameworks* de desenvolvimento e plataformas de gestão de dados, que contribuíram significativamente para a concretização deste trabalho.

Índice

Resumo (Abstract)	i
Agradecimentos	ii
1. Introdução	1
1.1. Contexto	1
1.2. Problema	1
1.3. Objetivos do Projeto	2
1.4. Contribuições do Projeto	2
2. Problema e Solução	3
2.1. Utilizadores-Alvo	3
2.2. Casos de Uso	3
3. Pipeline de Dados e Infraestrutura	6
3.1. Fontes de Dados	6
3.2. Ingestão de Dados	9
3.3. Persistência de Dados	9
4. Arquitetura de Inteligência (Neuro-Simbólica)	12
4.1. Abordagem Neuro-Simbólica	12
4.2. Agente de Seleção de Ferramentas	12
4.3. Ferramentas do Agente	13
4.4. API Backend e Fluxo de Execução	14
5. Modelagem C4	16
5.1. Contexto do Sistema (L1)	16
5.2. Containers (L2)	16
5.3. Componentes (L3)	17
5.4. Código (L4)	17
6. Interface de Suporte à Decisão	19
6.1. Dashboard Analítico	19
6.2. Chatbot Inteligente	19
6.3. Integração entre Chat e Dashboard	20
7. Processo de Desenvolvimento	21
7.1. Organização da Equipa	21
7.2. Workflow de Desenvolvimento	21
7.3. Planeamento Semanal	23
8. Implementação	24
8.1. Pipeline de Indexação	24
8.2. Pipeline de Consulta	25
8.3. Integração de Componentes	25
9. Avaliação e Métricas	27
9.1. Avaliação Automática (Startup Evaluation)	27
9.2. Avaliação Funcional na Interface de Utilizador (UI Evaluation)	30
9.3. Síntese da Avaliação	34
10. ESG e IA Responsável	35
10.1. Ética e Segurança	35
10.2. Governança e Transparência	35

10.3.	Sustentabilidade e Eficiência	35
11.	Conclusão e Trabalho Futuro	37
11.1.	Conclusão	37
11.2.	Trabalho Futuro	37
Anexo A.	ADRs — Decisões de Engenharia	38
A.A.	ADR-001 — Escolha da Base de Dados Vetorial	38
A.B.	ADR-002 - Escolha da base de dados estruturados	38
A.C.	ADR-002 - Escolha da base de dados semi-estruturados	39
A.D.	ADR-004 — Escolha do Modelo de Embeddings	40
A.E.	ADR-005 — Escolha do Modelo LLM	40
A.F.	ADR-006 — Escolha da API	40
A.G.	ADR-007 — Adoção da Framework LangChain para Consultas SQL	41
A.H.	ADR-008 — Implementação do Langfuse para Observabilidade e Tracing	42
A.I.	ADR-009 — Introdução de Makefile para Automação do Setup e Pipeline	42
A.J.	ADR-010 — Framework Streamlit para a Dashboard	43
A.K.	ADR-011 - Mudança para qwen2.5:1.5b	44

Índice de Figuras

Figura 1	Contexto do Sistema	16
Figura 2	Diagrama dos Containers	17
Figura 3	Diagrama de Componentes	17
Figura 4	Arquitetura do Sistema Desenvolvido	18
Figura 5	Gráfico de commits	22
Figura 6	Perguntas de Medicina Preventiva	31
Figura 7	Perguntas de Emergência Médica e sem Emergência Médica	32
Figura 8	Perguntas de Identidade do Sistema	33
Figura 9	Pergunta Fora de Contexto	33

1. Introdução

Neste capítulo apresenta-se o enquadramento geral do projeto, contextualizando o domínio em que se insere, identificando o problema que se pretende abordar e descrevendo os objetivos e contributos do sistema desenvolvido.

1.1. Contexto

O Serviço Nacional de Saúde (SNS) tem enfrentado, nos últimos anos, diversos desafios estruturais, resultantes principalmente do elevado volume de utentes que recorrem aos serviços de saúde, da escassez de profissionais em determinadas áreas e de limitações organizacionais que dificultam a gestão eficiente dos recursos disponíveis. Estas dificuldades manifestam-se frequentemente na sobrelotação dos serviços de urgência e em tempos de espera elevados para atendimento, o que compromete a qualidade e a rapidez da resposta prestada aos cidadãos.

Paralelamente, a crescente utilização de sistemas inteligentes na área da saúde tem vindo a ganhar relevância, em particular através da aplicação de técnicas de inteligência artificial e de análise de dados no apoio à organização da informação médica e à disponibilização de sistemas de suporte à decisão.

Neste contexto, os Sistemas de Suporte à Decisão baseados em conhecimento assumem um papel relevante, uma vez que permitem integrar informação proveniente de múltiplas fontes e disponibilizar mecanismos de exploração de conhecimento estruturado e não estruturado, apoiando a interpretação de informação no domínio da saúde.

A medicina preventiva desempenha igualmente um papel central neste tipo de sistemas, ao procurar promover a identificação precoce de sintomas e a disponibilização de informação educativa que contribua para a tomada de decisões mais informadas por parte dos utilizadores. Neste sentido, ferramentas computacionais podem contribuir para uma melhor compreensão de sintomas e doenças, ajudando a reduzir a utilização inadequada de serviços de urgência.

Neste contexto, o presente projeto propõe o desenvolvimento de um sistema de suporte à decisão baseado em conhecimento, sob a forma de um assistente inteligente, capaz de responder a questões relacionadas com sintomas, doenças e dados de saúde pública. O sistema integra múltiplas fontes de informação estruturadas e não estruturadas, funcionando como uma ferramenta de apoio informacional e educativo, e não como um sistema de diagnóstico clínico.

1.2. Problema

Um dos fatores que contribui para a elevada procura dos serviços de urgência hospitalar é a dificuldade que muitos utilizadores enfrentam na interpretação de sintomas e na avaliação da sua gravidade. Na ausência de orientação clara ou de acesso imediato a aconselhamento médico, é comum a utilização dos serviços de urgência em situações que poderiam ser resolvidas em cuidados de saúde primários ou através de medidas simples de autocuidado.

Esta situação contribui para a sobrecarga dos sistemas de saúde e dificulta a priorização de casos efetivamente urgentes. Adicionalmente, a falta de ferramentas acessíveis que permitam uma primeira análise informativa dos sintomas agrava este problema, sobretudo fora do horário de funcionamento dos serviços de saúde.

No entanto, o desenvolvimento de sistemas capazes de apoiar esta interpretação apresenta desafios significativos, nomeadamente a necessidade de integrar conhecimento estruturado (ex: relações sintoma–doença), dados estatísticos de saúde pública e literatura médica não estruturada, garantindo simultaneamente respostas consistentes, seguras e interpretáveis.

Deste modo, o sistema desenvolvido não pretende substituir a avaliação realizada por profissionais de saúde, mas sim atuar como uma ferramenta de apoio informacional e educativo, ajudando os utilizadores a compreender melhor possíveis relações entre sintomas e condições médicas, bem como a consultar informação relevante para decisões mais informadas.

1.3. Objetivos do Projeto

O objetivo geral deste projeto consiste no desenvolvimento de um Sistema de Suporte à Decisão baseado em conhecimento, assente numa arquitetura neuro-simbólica (NeSy), capaz de integrar múltiplas fontes de dados e apoiar a interpretação de informação no domínio da saúde preventiva.

Para atingir este objetivo, foram definidos os seguintes objetivos específicos:

- recolher e integrar dados provenientes de múltiplas fontes heterogéneas (APIs, datasets e documentos);
- implementar um pipeline de ingestão, processamento e armazenamento de dados em diferentes paradigmas (SQL, NoSQL e vetorial);
- desenvolver um motor de orquestração baseado em ferramentas (tools), combinando SQL, MongoDB, *Retrieval Augmented Generation* (RAG) e dashboards;
- implementar um mecanismo de recuperação de informação baseado em RAG com embeddings e reranking semântico;
- desenvolver um motor de regras determinístico para filtragem e resposta a casos simples antes da utilização de modelos generativos;
- disponibilizar um dashboard analítico interativo controlado por linguagem natural;
- garantir observabilidade e rastreabilidade do sistema através de logging estruturado e tracing de interações.

1.4. Contribuições do Projeto

O presente projeto apresenta várias contribuições relevantes no contexto da engenharia de sistemas de suporte à decisão baseados em dados.

Em primeiro lugar, destaca-se o desenvolvimento de uma arquitetura híbrida neuro-simbólica (NeSy), que combina mecanismos determinísticos (regras e consultas estruturadas) com modelos generativos baseados em LLMs.

Adicionalmente, o sistema integra um motor de orquestração de ferramentas (*tool-based routing*), permitindo a seleção dinâmica entre diferentes fontes de dados, nomeadamente PostgreSQL (dados estruturados), MongoDB (conhecimento semi-estruturado) e ChromaDB (recuperação semântica de documentos).

Outra contribuição relevante consiste na implementação de um pipeline de RAG com embeddings e reranking semântico, permitindo melhorar a qualidade e relevância das respostas geradas com base em literatura médica e documentos educativos.

O sistema inclui ainda um mecanismo de interação entre o utilizador e o dashboard analítico, no qual instruções em linguagem natural podem modificar dinamicamente o estado da visualização de dados.

Por fim, o projeto inclui uma camada de observabilidade e rastreamento de execução baseada em tracing de interações, bem como mecanismos de resiliência e degradação controlada quando componentes do sistema falham, garantindo robustez operacional.

2. Problema e Solução

Esta secção descreve o problema abordado pelo sistema e apresenta a solução proposta, incluindo a identificação dos principais utilizadores e a definição de cenários de utilização representativos das funcionalidades implementadas.

2.1. Utilizadores-Alvo

Os principais utilizadores do sistema são cidadãos que procuram compreender melhor os seus sintomas e obter orientação inicial relativamente a possíveis condições de saúde. Em particular, o sistema destina-se a utilizadores que pretendem interpretar sintomas, obter informação em linguagem acessível e perceber quais as medidas que podem adotar perante determinadas situações de saúde.

Adicionalmente, o sistema pode ser útil para utilizadores que procuram orientação em situações de saúde não urgentes, permitindo-lhes avaliar se os sintomas apresentados justificam a procura de assistência médica imediata ou se podem ser inicialmente geridos através de medidas simples de autocuidado. Desta forma, pretende-se contribuir para uma utilização mais informada e responsável dos serviços de saúde.

Importa salientar que o sistema não se destina a substituir a avaliação realizada por profissionais de saúde, mas sim a fornecer apoio informativo inicial que auxilie os utilizadores na tomada de decisões mais conscientes relativamente ao seu estado de saúde.

2.2. Casos de Uso

De forma a orientar o desenvolvimento do sistema e validar as suas funcionalidades, foram definidos diversos cenários de utilização que refletem situações reais de interação entre os utilizadores e o assistente inteligente. Estes cenários abrangem os diferentes tipos de conhecimento disponíveis na plataforma e exploram as várias capacidades oferecidas pela arquitetura neuro-simbólica implementada. Em particular, destaca-se o foco em medicina preventiva, que constitui o principal objetivo funcional do sistema.

2.2.1. Caso 1 — Medicina Preventiva e Promoção da Saúde

O cenário mais central do sistema está relacionado com a medicina preventiva e a promoção de estilos de vida saudáveis. Neste contexto, o utilizador procura informação sobre formas de reduzir o risco de doenças, adotar comportamentos preventivos ou compreender a relação entre hábitos de vida e determinadas condições clínicas.

Para responder a este tipo de pedidos, o sistema recorre a uma base de conhecimento baseada em *Retrieval Augmented Generation* (RAG), que integra literatura científica, recomendações da Organização Mundial da Saúde e conteúdos de cuidados de saúde domiciliários. Esta componente permite ao sistema fornecer respostas informativas e baseadas em evidência sobre prevenção de doenças e autocuidado.

Durante o desenvolvimento, foram testadas várias interações representativas deste cenário. Entre os exemplos considerados encontram-se perguntas como “*How can I prevent heart diseases?*”, “*How does physical activity reduce cardiovascular risk?*” e “*What are WHO guidelines for physical activity?*”, que avaliam a capacidade do sistema para explicar relações entre comportamento e risco clínico. Foram também incluídas questões mais práticas e orientadas ao autocuidado, como “*What is a home remedy*

to treat cough?”, permitindo validar a utilidade do sistema em contextos de sintomas ligeiros e cuidados básicos, que possam ser feitos facilmente em casa.

2.2.2. Caso 2 — Consulta de Informação Estruturada

Outro cenário relevante consiste na consulta de informação estruturada armazenada na base de dados relacional. Nestes casos, o utilizador procura obter dados concretos relacionados com doenças, sintomas ou medicamentos. Para responder a este tipo de pedidos, o sistema recorre à ferramenta SQL, que converte a questão colocada pelo utilizador em consultas executadas diretamente sobre a base de dados PostgreSQL.

Durante o desenvolvimento foram realizados vários testes deste tipo, com o objetivo de avaliar a capacidade do sistema para recuperar informação factual de forma precisa. Entre os exemplos considerados encontram-se perguntas como *“Show me 10 drugs to treat AIDS”*, *“Tell me all the medicines to treat acne”* e *“What are the side effects of doxycycline”*, que permitem avaliar a capacidade do sistema para recuperar informação farmacológica armazenada na base de dados. Foram também efetuados testes relacionados com a exploração do próprio conteúdo da base de dados, como a questão *“Using your SQL health database, how many distinct diseases are recorded, and list three disease names from the diseases table”*.

2.2.3. Caso 3 — Explicação de Conceitos Médicos

Outro cenário relevante consiste na obtenção de explicações médicas em linguagem acessível. Muitos utilizadores procuram compreender melhor doenças, sintomas ou conceitos clínicos sem recorrer a documentação excessivamente técnica. Nestes casos, o sistema utiliza a informação armazenada na base de dados documental, construída a partir de conteúdos provenientes da MedlinePlus e de metadados disponibilizados pela Organização Mundial da Saúde.

Foram realizados diversos testes para avaliar esta funcionalidade. Entre os exemplos utilizados encontram-se perguntas como *“What is diabetes and what are its symptoms?”*, *“Give me a summary of information about hypertension”* e *“Can you explain everything about asthma?”*. Foram ainda avaliadas versões simplificadas das mesmas questões, como *“In simple terms, what is asthma?”*, de forma a verificar a capacidade do sistema para adaptar o nível de detalhe e a linguagem utilizada. Adicionalmente, foram testadas consultas relacionadas com indicadores da Organização Mundial da Saúde, incluindo perguntas como *“Which WHO indicators are related to diabetes?”* e *“What data do you have regarding mental health?”*.

2.2.4. Caso 4 — Avaliação da Necessidade de Assistência Médica

Um dos objetivos do sistema consiste em fornecer orientação inicial relativamente ao nível de urgência de determinadas situações de saúde. Embora o sistema não realize diagnósticos médicos nem substitua a avaliação de profissionais de saúde, este deve ser capaz de identificar sinais potencialmente graves e encaminhar o utilizador para assistência adequada.

Para avaliar este comportamento foram considerados cenários representativos de diferentes níveis de gravidade. Um dos testes realizados utilizou a questão *“I fell down the stairs and my head is bleeding everywhere, what should I do?”*, onde o sistema deve reconhecer a potencial gravidade da situação e recomendar assistência médica imediata. Em contraste, foi igualmente testada uma situação menos preocupante através da pergunta *“I have been coughing and feeling cold today, should I go to the hospital?”*. Neste caso, espera-se que o sistema reconheça a ausência de sinais evidentes de emergência, fornecendo sugestões de autocuidado e indicando sinais de alerta que possam justificar avaliação médica posterior.

2.2.5. Caso 5 — Interação Conversacional e Gestão de Pedidos Fora do Domínio

Por fim, foram considerados cenários que não requerem acesso a qualquer fonte de dados específica e que permitem avaliar o comportamento geral do assistente enquanto agente conversacional. Estes testes são importantes para verificar se o sistema consegue explicar corretamente a sua identidade, finalidade e limitações.

Entre os exemplos utilizados encontram-se perguntas como *“Who are you?”* e *“What do you do?”*, através das quais se pretende validar a capacidade do sistema para descrever corretamente o seu papel enquanto assistente de apoio à decisão na área da saúde. Foram ainda considerados pedidos fora do domínio médico, como *“Who won the Super Bowl last year and what was the final score?”*, permitindo verificar se o sistema consegue reconhecer questões que não se enquadram no seu âmbito de atuação e responder de forma adequada.

3. Pipeline de Dados e Infraestrutura

Esta secção descreve o processo de recolha, processamento e armazenamento de dados utilizados pelo sistema. A arquitetura de dados desenvolvida tem como objetivo integrar informação proveniente de múltiplas fontes heterogéneas, permitindo suportar as diferentes funcionalidades do sistema.

3.1. Fontes de Dados

O sistema desenvolvido integra informação proveniente de múltiplas fontes heterogéneas, combinando dados estruturados, semi-estruturados e documentação científica. Esta diversidade de fontes permite suportar diferentes funcionalidades do sistema, desde a identificação de sintomas e a consulta de informação sobre doenças até à exploração de estatísticas de saúde pública e à obtenção de recomendações relacionadas com medicina preventiva.

De forma geral, as fontes de dados utilizadas podem ser agrupadas em três categorias principais: APIs públicas, datasets estruturados e documentos científicos recolhidos e preparados para recuperação semântica. Cada uma destas categorias desempenha um papel específico na arquitetura do sistema, contribuindo para os diferentes mecanismos de suporte à decisão disponibilizados pelo chatbot e pelo dashboard analítico.

Devido à dimensão dos ficheiros utilizados no projeto, os dados encontram-se disponíveis através do seguinte repositório partilhado no Google Drive:

https://drive.google.com/drive/folders/1V38_wH5ufAbp_uyb5OzacmbvVCrATZE0?usp=drive_link

3.1.1. APIs Públicas

Para complementar os datasets estáticos utilizados no projeto, foram integradas duas APIs públicas na área da saúde: a **Global Health Observatory (GHO) OData API**, disponibilizada pela Organização Mundial da Saúde (WHO), e a API **MedlinePlus**, disponibilizada pela National Library of Medicine (NIH).

A GHO API foi utilizada para recolher metadados relativos a indicadores globais de saúde pública. Em particular, foram obtidas informações sobre indicadores disponíveis, dimensões de análise e valores associados a essas dimensões. Estes dados são posteriormente armazenados em MongoDB, permitindo ao sistema responder a perguntas relacionadas com a descoberta de indicadores de saúde e com a estrutura dos dados disponibilizados pela Organização Mundial da Saúde.

Os principais endpoints utilizados foram:

1. <https://ghoapi.azureedge.net/api/Indicator>
2. <https://ghoapi.azureedge.net/api/Dimension>
3. <https://ghoapi.azureedge.net/api/DIMENSION/{dimension}/DimensionValues>

Importa referir que os dados armazenados correspondem apenas a metadados e não às séries temporais completas dos indicadores da WHO. Assim, esta fonte é utilizada sobretudo para exploração e contextualização de indicadores, e não para análises estatísticas quantitativas.

Adicionalmente, foi integrada a API **MedlinePlus**, utilizada para recolher resumos informativos sobre doenças e condições médicas. A partir dos nomes das doenças presentes na base de dados relacional, foram efetuadas consultas automáticas à API, sendo os resumos obtidos armazenados em MongoDB para posterior utilização pelo chatbot. Esta informação permite ao sistema responder a perguntas do tipo “What

is hypertension?” ou “What is asthma?”, fornecendo explicações acessíveis baseadas em fontes médicas reconhecidas.

3.1.2. Datasets Estruturados

Para suportar as funcionalidades de análise, exploração de dados e identificação de sintomas, foram integrados vários datasets estruturados provenientes de fontes públicas e repositórios de dados abertos. No total, o sistema incorpora mais de 1,7 milhões de registos estruturados distribuídos por diferentes domínios da saúde pública e da medicina preventiva.

3.1.2.1. Disease Symptom Description Dataset

O **Disease Symptom Description Dataset** constitui uma das principais fontes de conhecimento utilizadas pelo sistema para relacionar sintomas com possíveis condições médicas. Este dataset foi obtido através da plataforma Kaggle e contém informação relativa a doenças, sintomas, descrições clínicas e medidas de precaução.

Após o processo de transformação e normalização, foram armazenadas informações relativas a 41 doenças distintas, 133 sintomas e aproximadamente 4.920 relações entre doenças e sintomas. Adicionalmente, foram integrados conjuntos de recomendações e precauções associados a cada doença.

Esta fonte de dados suporta diretamente funcionalidades relacionadas com interpretação de sintomas, identificação de doenças associadas e apresentação de medidas preventivas ou recomendações iniciais aos utilizadores.

Dataset disponível em:

<https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset>

3.1.2.2. Behavioral Risk Factor Surveillance System (BRFSS)

O **Behavioral Risk Factor Surveillance System (BRFSS)** é um dos maiores sistemas de recolha de informação sobre saúde pública do mundo, sendo coordenado pelo **Centers for Disease Control and Prevention (CDC)** dos Estados Unidos.

O dataset utilizado corresponde à edição de 2023 e contém aproximadamente 433.323 respostas individuais. Entre os atributos disponíveis encontram-se indicadores relacionados com hábitos de vida, fatores de risco, doenças diagnosticadas, estado geral de saúde e medidas biométricas como peso, altura e índice de massa corporal.

Esta fonte de dados é particularmente relevante para a componente de medicina preventiva do sistema, permitindo explorar relações entre comportamentos, estilos de vida e diferentes condições de saúde.

Dataset disponível em:

<https://www.kaggle.com/datasets/isuruprabath/brfss-2023-csv-dataset>

3.1.2.3. Chronic Disease Indicators (CDI)

O dataset **Chronic Disease Indicators (CDI)** reúne um conjunto abrangente de indicadores relacionados com doenças crónicas na população dos Estados Unidos. Os dados são disponibilizados pelo CDC e incluem informação epidemiológica recolhida ao longo de vários anos.

Após ingestão, o sistema passou a dispor de aproximadamente 309.215 registos, abrangendo 19 tópicos distintos relacionados com doenças crónicas e 55 localizações geográficas correspondentes a estados e territórios norte-americanos.

Esta informação é utilizada principalmente pelo dashboard analítico, permitindo explorar tendências temporais, diferenças regionais e indicadores relacionados com doenças como diabetes, doenças cardiovasculares, cancro, asma ou problemas de saúde mental.

Dataset disponível em:

<https://catalog.data.gov/dataset/u-s-chronic-disease-indicators>

3.1.2.4. Global Health Statistics

O dataset **Global Health Statistics** fornece uma perspetiva internacional sobre diferentes indicadores de saúde pública. Este conjunto de dados contém aproximadamente 1.000.000 de registos distribuídos por 20 países, abrangendo o período entre 2000 e 2024.

Entre as variáveis disponíveis encontram-se indicadores epidemiológicos, taxas de prevalência, incidência e mortalidade, custos médios de tratamento, acesso a cuidados de saúde, indicadores económicos e métricas associadas à qualidade dos sistemas de saúde.

Esta fonte de dados é utilizada principalmente para alimentar o dashboard analítico e responder a perguntas quantitativas relacionadas com tendências globais de saúde.

Importa salientar que este dataset é sintético, tendo sido concebido para fins analíticos e educacionais. Consequentemente, os resultados obtidos a partir desta fonte devem ser interpretados como dados simulados e não como estatísticas oficiais.

Dataset disponível em:

<https://www.kaggle.com/datasets/malaiarasugraj/global-health-statistics>

3.1.2.5. WHO/UNICEF Immunization Coverage (WUENIC)

O sistema integra ainda os dados disponibilizados pelo programa **WHO/UNICEF Estimates of National Immunization Coverage (WUENIC)**. Esta fonte contém informação relativa à cobertura vacinal de diferentes países ao longo do tempo.

Após ingestão, foram armazenados aproximadamente 49.683 registos correspondentes a 194 países, 16 tipos de vacinas e dados históricos compreendidos entre 1997 e 2024.

A informação disponibilizada por esta fonte permite responder a questões relacionadas com taxas de vacinação, evolução temporal da cobertura vacinal e comparação entre diferentes países.

3.1.2.6. Drugs and Side Effects Dataset

Para suportar consultas relacionadas com medicamentos, efeitos secundários e tratamentos, foi integrado um dataset especializado contendo informação sobre medicamentos e respetivos efeitos adversos.

O conjunto de dados contém aproximadamente 2.931 registos, abrangendo mais de 2.900 medicamentos e 47 condições médicas distintas. Entre os atributos disponíveis encontram-se nomes comerciais e genéricos, efeitos secundários conhecidos, classes farmacológicas, classificações de segurança e avaliações efetuadas por utilizadores.

Esta fonte permite ao sistema responder a perguntas como **“What are the side effects of doxycycline?”** ou **“Show me drugs used to treat acne”**.

3.1.3. Recolha de Literatura Científica

Para complementar os dados estruturados, foi desenvolvido um mecanismo de recolha e preparação de literatura científica relacionada com medicina preventiva.

Inicialmente foram exploradas diferentes fontes de documentação científica, incluindo motores de pesquisa acadêmicos e bases de dados biomédicas. Após a avaliação das alternativas disponíveis, optou-se por utilizar principalmente conteúdos provenientes do **PubMed Central (PMC)** e documentos disponibilizados por organizações de referência na área da saúde.

Os documentos recolhidos abordam temas relacionados com promoção da saúde, prevenção de doenças crónicas, vacinação, rastreios médicos, atividade física, nutrição e outros tópicos relevantes para a medicina preventiva.

Para além da literatura científica, foi igualmente integrado um documento especializado sobre cuidados de saúde domiciliários e remédios caseiros, designado **The Quick Guide to Home Remedies**. Este documento complementa a componente preventiva do sistema, permitindo responder a questões relacionadas com medidas simples de autocuidado e gestão de sintomas ligeiros.

Após a recolha, os documentos são processados, segmentados em múltiplos fragmentos de texto e preparados para posterior geração de embeddings. Estes fragmentos constituem a base de conhecimento utilizada pelo mecanismo de recuperação semântica implementado no sistema, permitindo enriquecer as respostas geradas pelo chatbot com informação proveniente de literatura científica relevante.

3.2. Ingestão de Dados

A ingestão de dados constitui uma das componentes centrais da arquitetura do sistema, sendo responsável pela recolha, transformação e armazenamento das diferentes fontes de informação utilizadas pelo chatbot e pelo dashboard analítico.

Devido à diversidade das fontes integradas, foi adotada uma abordagem baseada em processos **ETL (Extract, Transform, Load)**, permitindo uniformizar a forma como os dados são recolhidos e preparados para utilização posterior. Todas as etapas de ingestão foram implementadas em Python e organizadas em módulos independentes, facilitando a manutenção e a expansão futura da plataforma.

Numa primeira fase, os dados são extraídos das respetivas fontes. Esta etapa inclui a leitura de datasets em formato CSV, a realização de pedidos a APIs públicas e a recolha de documentos científicos destinados à componente de recuperação semântica. Cada fonte possui um pipeline de ingestão específico, adaptado ao formato e às características dos dados disponibilizados.

Após a extração, os dados passam por um conjunto de transformações destinadas a garantir a sua consistência e qualidade. Entre as operações realizadas encontram-se a remoção de duplicados, a normalização de nomes e formatos, a validação de campos obrigatórios e a adaptação das estruturas de dados ao modelo de armazenamento pretendido.

Finalmente, os dados transformados são carregados para os diferentes sistemas de persistência utilizados pelo projeto. Dados estruturados são armazenados em PostgreSQL, informação semi-estruturada é armazenada em MongoDB e conteúdos textuais destinados à recuperação semântica são indexados em ChromaDB através da geração de embeddings.

A execução da pipeline de ingestão é realizada através do script principal **run_ingestion.py**, responsável por coordenar os diferentes módulos de ingestão e garantir que todas as bases de dados são populadas de forma consistente. Esta abordagem simplifica a preparação do ambiente e permite reconstruir toda a infraestrutura de dados a partir das fontes originais sempre que necessário.

3.3. Persistência de Dados

A arquitetura de persistência foi concebida segundo uma abordagem poliglota (**polyglot persistence**), recorrendo a diferentes tecnologias de armazenamento em função das características dos dados e dos padrões de acesso pretendidos.

Dado que o sistema integra simultaneamente dados estruturados, documentos semi-estruturados e informação textual destinada a pesquisa semântica, não seria eficiente utilizar uma única tecnologia de armazenamento para todos os casos. Assim, foram adotadas três soluções complementares: PostgreSQL para dados relacionais, MongoDB para documentos JSON e ChromaDB para armazenamento vetorial.

3.3.1. Base de Dados SQL

Para o armazenamento de dados estruturados foi utilizada uma base de dados relacional baseada em PostgreSQL. Esta componente constitui a principal fonte de informação para consultas analíticas, exploração estatística e obtenção de dados estruturados por parte do agente inteligente.

Atualmente, a base de dados integra informação proveniente de múltiplos datasets relacionados com saúde pública, sintomas, doenças, vacinação, fatores de risco e medicamentos.

O conjunto de tabelas associado ao **Disease Symptom Description Dataset** encontra-se organizado segundo um modelo relacional normalizado. A tabela **diseases** armazena as diferentes doenças disponíveis, enquanto a tabela **symptoms** contém os sintomas associados. A relação entre ambas é implementada através da tabela intermédia **disease_symptoms**, permitindo representar associações do tipo muitos-para-muitos. Adicionalmente, a tabela **disease_precautions** armazena recomendações e medidas preventivas associadas a cada condição médica.

O dataset BRFSS foi integrado através da tabela **brfss_responses**, contendo aproximadamente 433 mil registos relacionados com fatores de risco, hábitos de vida e diagnósticos médicos. Esta informação é particularmente relevante para a componente de medicina preventiva do sistema.

Os indicadores epidemiológicos provenientes do dataset **Chronic Disease Indicators** foram armazenados na tabela **chronic_disease_indicators**, contendo mais de 300 mil registos relativos a doenças crónicas e respetivos indicadores estatísticos.

O dataset **Global Health Statistics** encontra-se representado na tabela **global_health_stats**, contendo aproximadamente um milhão de registos com indicadores globais de saúde, fatores socioeconómicos e métricas relacionadas com acesso a cuidados médicos.

Os dados relativos à cobertura vacinal foram organizados através das tabelas **country_dim**, **vaccine_dim** e **immunization_fact**, permitindo modelar adequadamente as relações entre países, vacinas e observações temporais.

Por fim, foi integrada a tabela **drugs_side_effects**, responsável pelo armazenamento de informação relativa a medicamentos, condições médicas tratadas, efeitos secundários e outras características farmacológicas relevantes.

A utilização de PostgreSQL permite ao sistema executar consultas estruturadas de forma eficiente, sendo esta a principal fonte utilizada pela ferramenta SQL disponibilizada ao agente inteligente.

3.3.2. Base de Dados NoSQL

Para o armazenamento de informação semi-estruturada foi utilizada uma base de dados orientada a documentos baseada em MongoDB.

Esta componente é utilizada para armazenar conteúdos provenientes de APIs externas e documentos cuja estrutura não se adapta naturalmente a um modelo relacional. A flexibilidade oferecida pelo MongoDB permite preservar a estrutura original dos dados e simplificar futuras extensões da plataforma.

Uma das principais utilizações desta base de dados consiste no armazenamento dos metadados obtidos através da **Global Health Observatory API**. Os indicadores, dimensões e respetivos valores são armazenados sob a forma de documentos JSON, permitindo ao sistema responder a questões relacionadas com indicadores da Organização Mundial da Saúde.

Adicionalmente, o MongoDB é utilizado para armazenar informação proveniente da API **MedlinePlus**. Os documentos recolhidos incluem descrições de doenças, sintomas associados, causas conhecidas e outras informações médicas relevantes. Estes conteúdos são posteriormente utilizados pelo chatbot para gerar explicações acessíveis e contextualizadas sobre diferentes condições médicas.

A utilização de MongoDB revelou-se particularmente adequada para este tipo de informação, uma vez que diferentes doenças podem apresentar estruturas documentais distintas e níveis variáveis de detalhe.

As coleções existentes encontram-se organizadas por domínio funcional, permitindo separar claramente a informação proveniente da WHO da informação recolhida através da MedlinePlus.

3.3.3. Base de Dados Vetorial

Para suportar a componente de **Retrieval-Augmented Generation (RAG)** foi utilizada uma base de dados vetorial baseada em ChromaDB.

Esta tecnologia permite armazenar representações vetoriais (**embeddings**) de fragmentos de texto, possibilitando a realização de pesquisas semânticas baseadas no significado das perguntas colocadas pelos utilizadores e não apenas na correspondência exata de palavras.

Os documentos científicos recolhidos durante o processo de ingestão são inicialmente divididos em múltiplos segmentos de texto (**chunks**). Cada segmento é posteriormente convertido para uma representação vetorial através do modelo de embeddings **BAAI/bge-base-en-v1.5**.

A escolha deste modelo deve-se ao seu elevado desempenho em tarefas de recuperação semântica e à sua capacidade para representar adequadamente terminologia biomédica e conceitos relacionados com medicina preventiva.

Após a geração dos embeddings, os vetores são armazenados em ChromaDB juntamente com metadados relevantes, incluindo a origem do documento, o título e outros identificadores necessários para rastrear a proveniência da informação recuperada.

Atualmente, a base de dados vetorial contém várias centenas de fragmentos de texto provenientes de artigos científicos e documentos especializados sobre medicina preventiva, atividade física, doenças cardiovasculares, vacinação, rastreios médicos e remédios caseiros.

Quando um utilizador coloca uma questão relacionada com literatura científica ou práticas preventivas, o sistema realiza uma pesquisa semântica em ChromaDB para identificar os fragmentos mais relevantes. Esses conteúdos são posteriormente utilizados pelo modelo de linguagem para gerar respostas fundamentadas em conhecimento científico.

Esta componente constitui um elemento essencial da arquitetura neuro-simbólica implementada no projeto, permitindo combinar as capacidades generativas do modelo de linguagem com conhecimento factual proveniente de fontes científicas externas.

4. Arquitetura de Inteligência (Neuro-Simbólica)

Esta secção descreve a arquitetura de inteligência artificial implementada no sistema, bem como os diferentes componentes responsáveis pela interpretação das perguntas dos utilizadores, seleção das fontes de conhecimento adequadas e geração das respostas apresentadas pelo chatbot.

4.1. Abordagem Neuro-Simbólica

Os sistemas neuro-simbólicos (**Neuro-Symbolic Systems** ou **NeSy**) combinam componentes neurais, responsáveis pelo processamento e geração de linguagem natural, com componentes simbólicos baseados em regras, lógica e conhecimento estruturado. Esta abordagem procura tirar partido das capacidades de raciocínio e flexibilidade dos modelos de linguagem, complementando-as com mecanismos determinísticos que garantem maior controlo, transparência e fiabilidade das respostas produzidas.

No contexto deste projeto, a componente neuronal é representada por um **Large Language Model** (LLM) executado localmente através da plataforma Ollama. Durante a fase de desenvolvimento foram avaliados diferentes modelos da família Gemma e Qwen, nomeadamente **Gemma 3 1B**, **Gemma 3 4B** e **Qwen 2.5 1.5B**.

Os testes realizados tiveram como principal objetivo avaliar a capacidade dos modelos para interpretar perguntas médicas, selecionar corretamente ferramentas especializadas, compreender instruções complexas e produzir respostas consistentes em ambiente local. Embora o modelo **Gemma 3 4B** apresentasse respostas geralmente mais detalhadas, os seus requisitos computacionais e tempos de inferência revelaram-se significativamente superiores. Por outro lado, o modelo **Gemma 3 1B** demonstrou maior rapidez, mas apresentou menor consistência na seleção de ferramentas e na interpretação de determinadas instruções.

Após a fase de experimentação, foi adotado o modelo **Qwen 2.5 1.5B** como componente principal do sistema. Este modelo apresentou o melhor equilíbrio entre qualidade das respostas, velocidade de execução e consumo de recursos computacionais, demonstrando simultaneamente maior robustez na interação com os restantes componentes da arquitetura.

A componente simbólica é composta por um conjunto de regras determinísticas, mecanismos de seleção de ferramentas e sistemas de acesso a conhecimento estruturado. Estes componentes permitem complementar o conhecimento paramétrico do modelo com informação factual proveniente de bases de dados e documentos científicos, reduzindo a probabilidade de alucinações e aumentando a fiabilidade das respostas produzidas.

4.2. Agente de Seleção de Ferramentas

Um dos componentes centrais da arquitetura é o agente responsável pela seleção dinâmica de ferramentas. Este agente atua como um mecanismo de orquestração que analisa cada pergunta colocada pelo utilizador e determina quais os componentes mais adequados para produzir a resposta.

Após ultrapassar a camada inicial de regras determinísticas, a pergunta é enviada para o agente de seleção, que utiliza o modelo de linguagem para identificar a intenção do utilizador e selecionar uma ou mais ferramentas especializadas. Dependendo do tipo de questão colocada, o sistema pode recorrer exclusivamente a uma ferramenta ou combinar múltiplas fontes de informação numa única resposta.

Por exemplo, perguntas relacionadas com estatísticas ou medicamentos são normalmente encaminhadas para a ferramenta SQL, enquanto perguntas sobre indicadores da Organização Mundial da Saúde utilizam a ferramenta MongoDB. Questões relacionadas com literatura científica ou medicina preventiva recorrem à ferramenta RAG. Em determinados cenários, o sistema pode ainda combinar várias ferramentas simultaneamente, agregando os resultados obtidos antes da geração da resposta final.

Esta abordagem permite separar claramente a lógica de decisão da lógica de acesso aos dados, tornando o sistema mais modular, extensível e fácil de manter.

4.3. Ferramentas do Agente

O agente inteligente dispõe de um conjunto de ferramentas especializadas, cada uma otimizada para um tipo específico de conhecimento ou operação.

4.3.1. SQL Tool

A ferramenta SQL permite consultar a base de dados relacional PostgreSQL, onde se encontram armazenados os principais datasets estruturados do sistema.

Esta ferramenta é utilizada para responder a perguntas relacionadas com sintomas, doenças, medicamentos, efeitos secundários, estatísticas epidemiológicas, vacinação e indicadores de saúde pública. Através desta ferramenta, o sistema consegue responder a questões quantitativas e factuais que exigem acesso direto aos dados armazenados.

Entre os exemplos de perguntas suportadas encontram-se a identificação de medicamentos associados a uma determinada condição médica, a consulta de efeitos secundários conhecidos, a obtenção de estatísticas de saúde pública ou a exploração de relações entre sintomas e doenças.

A utilização de uma base de dados relacional permite garantir respostas precisas e fundamentadas em dados estruturados, reduzindo a dependência exclusiva do conhecimento interno do modelo de linguagem.

4.3.2. MongoDB Tool

A ferramenta MongoDB permite consultar a base de dados orientada a documentos utilizada pelo sistema.

Esta ferramenta é responsável por aceder a informação proveniente de duas fontes principais. A primeira corresponde aos metadados recolhidos através da **Global Health Observatory API** da Organização Mundial da Saúde, incluindo indicadores, dimensões e classificações relacionadas com saúde pública. A segunda corresponde aos documentos obtidos através da API MedlinePlus, que incluem descrições de doenças, sintomas, causas e informação educativa direcionada para o público em geral.

Graças a esta ferramenta, o sistema consegue responder a perguntas como a identificação de indicadores da WHO relacionados com uma determinada doença ou fornecer explicações acessíveis sobre condições médicas específicas.

A utilização de MongoDB permite preservar a estrutura original dos documentos recolhidos, oferecendo simultaneamente elevada flexibilidade para futuras expansões da plataforma.

4.3.3. RAG Tool

A ferramenta RAG (**Retrieval-Augmented Generation**) constitui o principal mecanismo de acesso à literatura científica utilizada pelo sistema.

Quando uma pergunta exige conhecimento proveniente de artigos científicos ou documentos especializados, o sistema realiza uma pesquisa semântica sobre os conteúdos armazenados em ChromaDB. Os fragmentos de texto mais relevantes são recuperados e posteriormente utilizados para complementar o contexto fornecido ao modelo de linguagem.

Esta ferramenta é particularmente importante para a vertente de medicina preventiva do projeto. Através dela, o sistema consegue responder a perguntas relacionadas com prevenção de doenças, benefícios da atividade física, fatores de risco cardiovasculares, rastreios médicos, vacinação e outras recomendações fundamentadas em literatura científica.

Adicionalmente, esta ferramenta permite consultar informação proveniente do documento **The Quick Guide to Home Remedies**, possibilitando responder a questões relacionadas com medidas simples de autocuidado e remédios caseiros para situações não urgentes.

A integração de RAG permite reduzir a dependência do conhecimento paramétrico do modelo de linguagem, fornecendo respostas suportadas por informação externa explicitamente recuperada durante a execução.

4.3.4. Dashboard Tool

A ferramenta de dashboard permite ao chatbot interagir diretamente com o painel analítico da aplicação.

Sempre que o utilizador solicita a visualização de determinados dados ou a aplicação de filtros específicos, esta ferramenta é responsável por interpretar a intenção da pergunta e atualizar o estado do dashboard. Desta forma, o sistema cumpre um dos requisitos definidos para o projeto, permitindo que o chatbot não funcione apenas como uma interface conversacional, mas também como um mecanismo de controlo da visualização analítica.

Entre as operações suportadas encontram-se a aplicação de filtros geográficos, temporais e temáticos sobre os diferentes conjuntos de dados disponibilizados no dashboard.

4.4. API Backend e Fluxo de Execução

A arquitetura de inteligência encontra-se integrada numa API **backend** desenvolvida em Python, responsável por coordenar a comunicação entre o frontend, o modelo de linguagem, as ferramentas especializadas e as bases de dados.

O principal ponto de entrada da aplicação corresponde ao endpoint responsável por processar as mensagens enviadas pelo utilizador. Sempre que uma nova pergunta é recebida, o sistema inicia um fluxo de execução composto por várias etapas.

Numa primeira fase, a pergunta é analisada pelo motor de regras determinísticas. Este componente verifica se a questão corresponde a cenários previamente definidos, como situações de emergência médica, perguntas frequentes sobre o sistema ou questões claramente fora do domínio da aplicação. Quando existe correspondência, a resposta é produzida diretamente sem necessidade de recorrer ao modelo de linguagem.

Caso nenhuma regra seja ativada, a pergunta é encaminhada para o agente de seleção de ferramentas. Este componente analisa a intenção do utilizador e determina quais as ferramentas mais adequadas para responder à questão.

As ferramentas selecionadas são posteriormente executadas, obtendo informação proveniente das diferentes bases de dados e sistemas de recuperação de informação. Quando necessário, os resultados provenientes de múltiplas ferramentas são agregados numa única estrutura de contexto.

Por fim, o modelo de linguagem utiliza a informação recolhida para gerar a resposta final apresentada ao utilizador. Este fluxo híbrido permite combinar a flexibilidade dos modelos de linguagem com a precisão

dos sistemas de consulta estruturada e recuperação de conhecimento, constituindo o núcleo da arquitetura neuro-simbólica implementada no projeto.

5. Modelagem C4

Este capítulo apresenta a modelação da arquitetura do sistema utilizando o modelo C4. Este modelo permite descrever o sistema em diferentes níveis de abstracção, desde a visão global até à implementação detalhada, facilitando a compreensão da sua estrutura e dos seus principais componentes.

5.1. Contexto do Sistema (L1)

O *System Context* representa o sistema DrHouseGPT no seu ambiente externo, identificando o utilizador final e as principais dependências externas. O sistema atua como um intermediário entre o utilizador e diferentes fontes de dados, incluindo bases de dados relacionais, bases de dados documentais, motores de pesquisa semântica e um modelo de linguagem local (ver [Figura 1](#)).

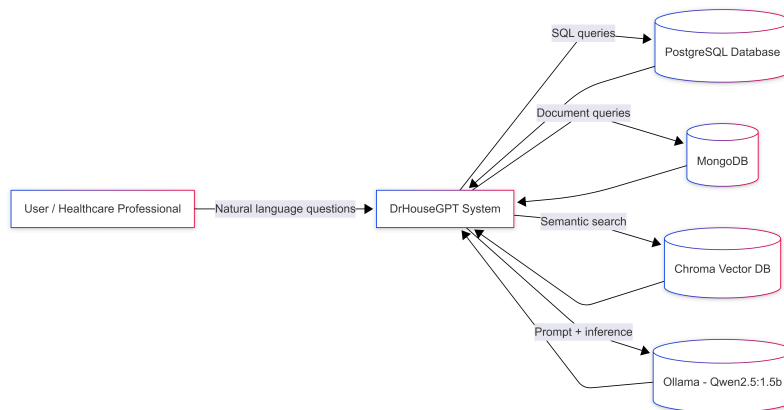


Figura 1: Contexto do Sistema

5.2. Containers (L2)

O *Container Diagram* descreve a decomposição do sistema em componentes de alto nível, incluindo a aplicação de interface, o agente de orquestração, as ferramentas de processamento e as bases de dados utilizadas. Esta divisão permite separar responsabilidades e garantir escalabilidade da solução (ver [Figura 2](#)).

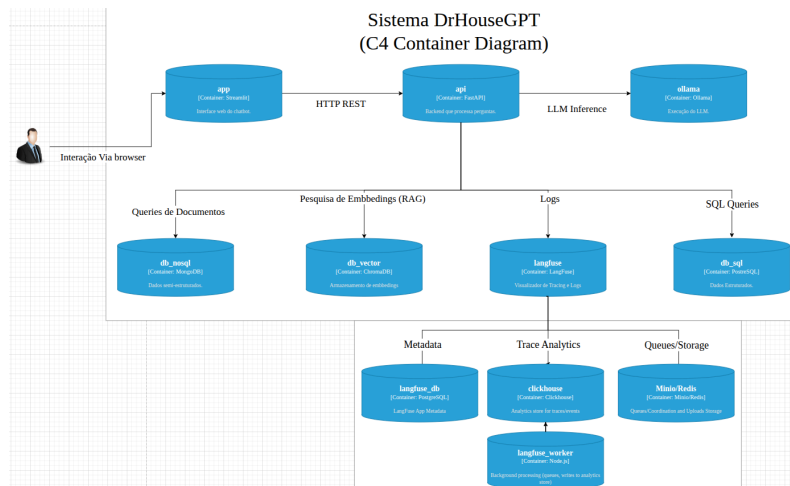


Figura 2: Diagrama dos Containers

5.3. Componentes (L3)

O *Component Diagram* detalha a estrutura interna do sistema, destacando o agente responsável pela seleção de ferramentas e os diferentes módulos especializados, como SQL Tool, RAG Pipeline e MongoDB Retriever. Cada componente interage com diferentes fontes de dados e contribui para a geração da resposta final (ver [Figura 3](#)).

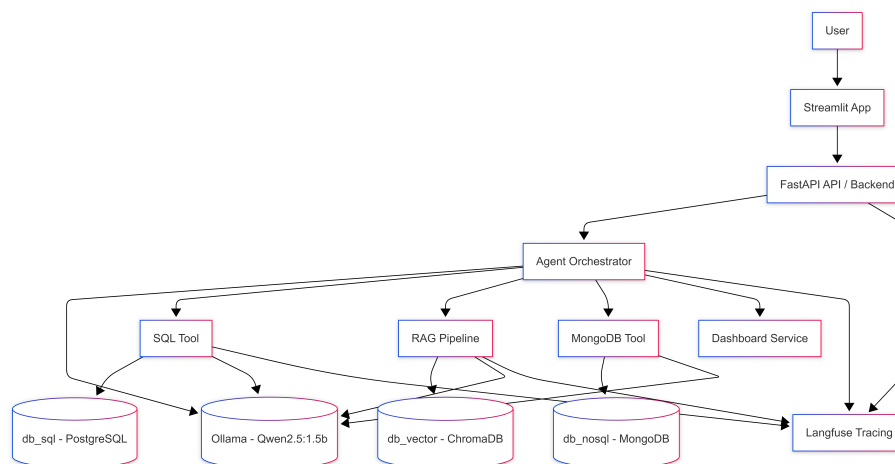


Figura 3: Diagrama de Componentes

5.4. Código (L4)

O nível de implementação reflete os principais módulos desenvolvidos no sistema, incluindo a pipeline de RAG, a ferramenta de geração de queries SQL e a integração com a dashboard. Estes componentes concretizam a arquitetura definida nos níveis superiores (ver [Figura 4](#)).

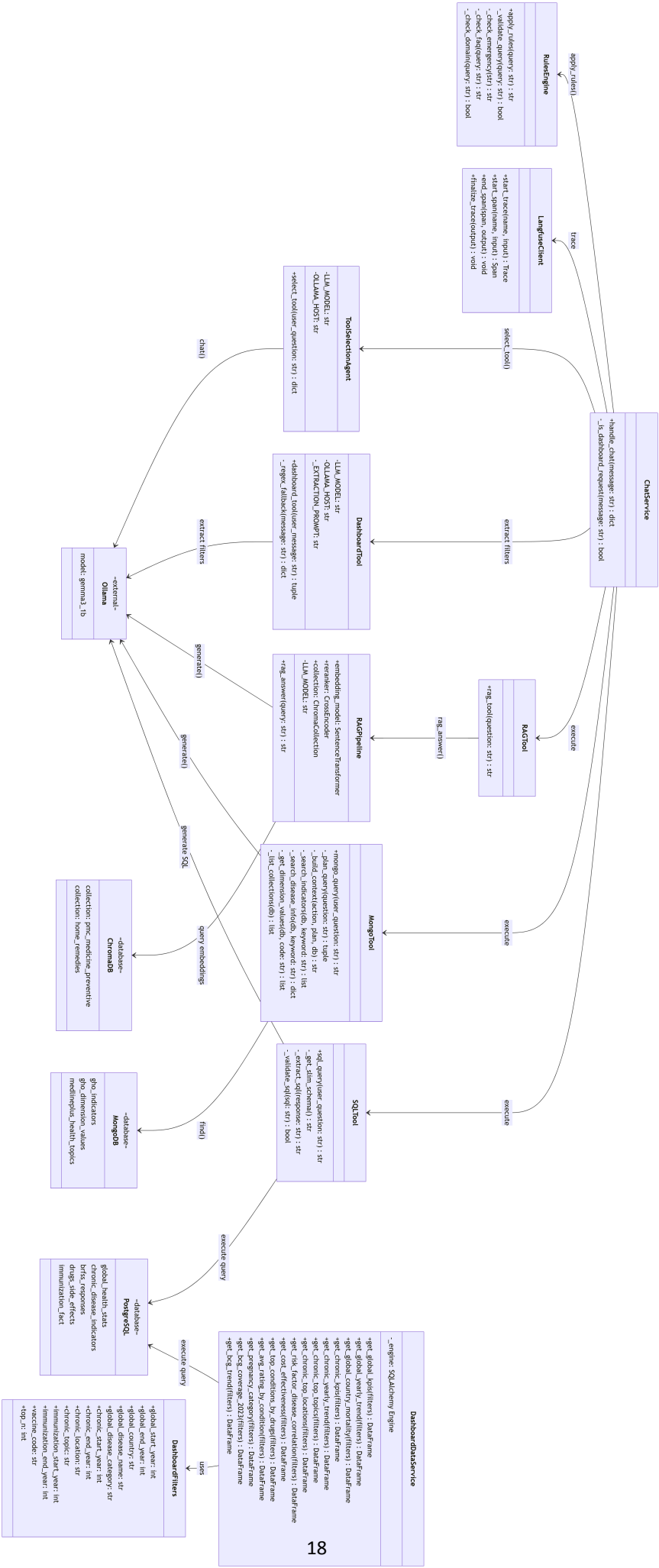


Figura 4: Arquitetura do Sistema Desenvolvido

6. Interface de Suporte à Decisão

Esta secção descreve a interface desenvolvida para a interação do utilizador com o sistema, composta por um dashboard analítico e um chatbot inteligente que funcionam de forma integrada numa única aplicação web.

6.1. Dashboard Analítico

A interface visual foi desenvolvida com o framework Streamlit, organizada em cinco secções temáticas que agregam diferentes domínios de dados de saúde.

A secção Global Overview apresenta um mapa coroplético interativo com projeção ortográfica, onde cada país é colorido de acordo com a taxa de mortalidade média. Acompanha este mapa um gráfico de linha com a evolução temporal das taxas de mortalidade e recuperação ao longo dos anos disponíveis. No topo da página são apresentados três indicadores principais: taxa de mortalidade média, taxa de recuperação média e número de países em análise.

A secção Chronic Disease é dedicada aos dados do CDC para os Estados Unidos, com base no dataset Chronic Disease Indicators. Apresenta três KPIs relativos ao número de indicadores, localizações cobertas e valor médio dos indicadores, complementados por um gráfico de tendência anual com intervalo de confiança, um ranking dos tópicos com maior valor médio e um ranking dos estados americanos mais afetados.

A secção Risk and Cost combina duas visualizações distintas. A primeira é um mapa de calor que cruza fatores de risco comportamentais (tabagismo, pressão arterial, colesterol, consumo de álcool, sedentarismo e IMC) com prevalências de doenças diagnosticadas, construído a partir dos dados do inquérito BRFSS. A segunda é um gráfico de barras que mostra a proporção da população com cada fator de risco, para o estado selecionado. Um gráfico de dispersão relaciona o custo médio de tratamento com a taxa de recuperação por doença e país, permitindo identificar as intervenções com melhor relação custo-eficácia.

A secção Drug Insights apresenta três gráficos relativos à base de dados de medicamentos: os medicamentos mais revistos por condição médica, a classificação média por condição e a distribuição das categorias de segurança na gravidez segundo a FDA.

A secção Immunization mostra a cobertura vacinal por país para o ano selecionado e a evolução temporal da cobertura média global, com valores mínimos e máximos anuais.

6.2. Chatbot Inteligente

O chatbot é acessível num painel lateral à esquerda da aplicação. O utilizador escreve uma pergunta em linguagem natural e recebe uma resposta gerada pelo sistema, sem necessidade de conhecer a estrutura dos dados subjacentes.

Quando uma mensagem é submetida, é enviada para a API REST desenvolvida em FastAPI. O componente ChatService recebe o pedido e executa um pipeline sequencial. Em primeiro lugar, é verificado se a mensagem constitui um pedido de alteração do dashboard, caso em que o processamento é imediato e não envolve o motor de regras. Caso contrário, o motor de regras valida a mensagem quanto ao comprimento mínimo, deteta situações de emergência médica, verifica correspondência com perguntas frequentes e confirma que a pergunta se enquadra no domínio médico. Se a mensagem passar estas verificações, um agente de seleção de ferramentas consulta o modelo de linguagem local para decidir qual a ferramenta

mais adequada: pesquisa semântica por similaridade vetorial (RAG), consulta a dados estruturados em PostgreSQL via SQL gerado automaticamente, consulta a indicadores da Organização Mundial de Saúde em MongoDB, ou extração de filtros para o dashboard.

O modelo de linguagem utilizado é o **Qwen 2.5 1.5B**, servido localmente através do Ollama, garantindo que nenhum dado do utilizador é enviado para serviços externos. Todas as operações são registadas com traces e spans no Langfuse, permitindo monitorizar o desempenho e o comportamento do sistema em produção.

6.3. Integração entre Chat e Dashboard

A integração entre o chat e o dashboard é o elemento central desta interface. Quando o utilizador formula um pedido de alteração do dashboard em linguagem natural, o sistema extrai automaticamente os filtros pretendidos e atualiza todas as visualizações em tempo real, sem que o utilizador precise de interagir diretamente com qualquer elemento gráfico.

O componente DashboardTool é responsável por esta extração. Utiliza o modelo de linguagem para interpretar a mensagem e produzir um conjunto de filtros estruturados. Quando o modelo não consegue produzir uma resposta válida, um mecanismo de fallback baseado em expressões regulares garante que os casos mais comuns são igualmente tratados. Os filtros disponíveis cobrem o nome da doença, o país, o intervalo de anos, a categoria de doença, o estado americano, o tópico de doença crónica e o código de vacina.

Quando a API devolve filtros ao cliente, a aplicação Streamlit combina-os com o estado existente da sessão e aciona uma nova renderização do dashboard. As consultas às bases de dados são executadas novamente com os parâmetros atualizados, e todos os gráficos refletem os novos dados de forma sincronizada.

A título de exemplo, um utilizador pode escrever “show me diabetes data for Brazil from 2015 to 2020” e o sistema atualiza simultaneamente o mapa global, o gráfico de tendência, os gráficos de medicamentos e os indicadores de topo para refletir exatamente esse contexto. Da mesma forma, “filter chronic data to California” restringe a secção de doenças crónicas e o mapa de fatores de risco ao estado da Califórnia.

7. Processo de Desenvolvimento

Esta secção descreve como o projeto foi organizado e desenvolvido ao longo do semestre.

7.1. Organização da Equipa

A equipa de desenvolvimento foi composta por vários elementos com responsabilidades distribuídas por diferentes áreas do projeto, incluindo inteligência artificial, engenharia de dados, desenvolvimento *backend* e gestão do projeto.

Ao longo do desenvolvimento, os membros assumiram papéis como:

- *AI Engineer* : responsável pela integração do modelo de linguagem, *prompts*, ferramentas inteligentes e testes ao *chatbot*.
- *Data Engineer*: responsável pela ingestão, transformação e organização dos dados.
- *Backend Engineer*: responsável pela API, integração entre serviços e lógica do sistema.
- *Project Master* : responsável pela coordenação geral, planeamento semanal e acompanhamento das tarefas.

Apesar da existência de responsabilidades principais, o projeto foi desenvolvido de forma colaborativa, existindo rotação parcial de tarefas entre os membros da equipa e obrigando também com que os papéis não fossem estáticos ao longo do desenvolvimentos do projeto. Esta abordagem permitiu maior partilha de conhecimento, melhor compreensão global da arquitetura do sistema e maior flexibilidade no desenvolvimento.

Ao longo do projeto, diferentes membros contribuíram em múltiplas componentes, *frontend*, *dashboards*, ferramentas inteligentes, *pipelines* de dados e testes ao sistema.

7.2. Workflow de Desenvolvimento

O desenvolvimento do projeto seguiu práticas modernas de engenharia de software e colaboração em equipa.

7.2.1. Git e GitHub

O controlo de versões do projeto foi realizado através do Git, utilizando o GitHub como plataforma central de alojamento do repositório e colaboração entre os membros da equipa.

A utilização de um sistema de controlo de versões revelou-se fundamental para garantir a organização e rastreabilidade do desenvolvimento, permitindo acompanhar todas as alterações efetuadas ao longo do projeto, identificar rapidamente a origem de problemas e recuperar versões anteriores quando necessário.

A estratégia adotada baseou-se na utilização de branches dedicadas para cada funcionalidade ou tarefa em desenvolvimento. Esta abordagem permitiu que diferentes elementos da equipa trabalhassem em simultâneo sem interferirem diretamente no trabalho dos restantes membros, reduzindo conflitos e facilitando a gestão do código-fonte.

Além disso, o GitHub serviu como ponto central de partilha de conhecimento, documentação e acompanhamento da evolução do projeto, funcionando como uma ferramenta essencial para a coordenação das atividades da equipa.

7.2.2. Pull Requests e Revisão de código

A integração de novas funcionalidades no branch principal foi realizada através de pull requests, permitindo uma validação prévia antes da incorporação definitiva do código.

Sempre que possível, as alterações submetidas eram analisadas por outros membros da equipa, promovendo um processo informal de revisão de código. Esta prática contribuiu para a deteção precoce de erros, identificação de melhorias de implementação e uniformização das práticas de desenvolvimento adotadas ao longo do projeto.

As pull requests permitiram ainda documentar as alterações realizadas, associando cada conjunto de modificações a uma funcionalidade específica ou correção de problemas identificados durante o desenvolvimento.

A utilização conjunta de branches, pull requests e revisão de código proporcionou um processo de desenvolvimento mais estruturado, facilitando a colaboração entre os membros da equipa e contribuindo para a qualidade e manutenção do sistema desenvolvido.

A [Figura 5](#) apresenta a evolução temporal dos commits realizados ao longo do desenvolvimento do projeto. É possível observar uma atividade contínua durante o semestre, refletindo a implementação progressiva das diferentes componentes do sistema, incluindo o pipeline de dados, os agentes, o mecanismo RAG, a dashboard analítica e as integrações com as diversas bases de dados.

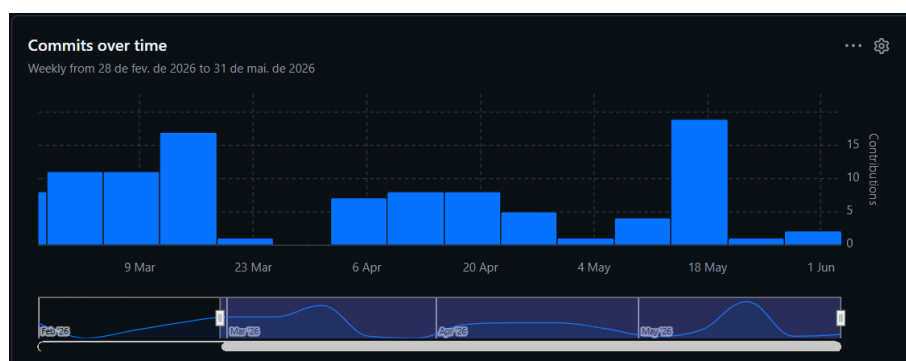


Figura 5: Gráfico de commits

7.2.3. Docker

O sistema foi totalmente containerizado através de Docker, permitindo encapsular todos os componentes necessários ao funcionamento da aplicação em ambientes isolados e reproduzíveis.

A utilização de containers facilitou significativamente o desenvolvimento colaborativo, uma vez que todos os elementos da equipa passaram a utilizar exatamente as mesmas versões de bibliotecas, dependências e serviços, eliminando problemas associados a diferenças entre sistemas operativos ou configurações locais.

A orquestração dos serviços foi realizada através do Docker Compose, responsável por gerir o ciclo de vida dos vários componentes da plataforma. Entre estes componentes destacam-se a API backend desenvolvida em Python, o frontend da aplicação, as bases de dados PostgreSQL, MongoDB e ChromaDB, bem como o serviço Ollama responsável pela execução local dos modelos de linguagem utilizados pelo sistema.

A comunicação entre os diferentes containers é realizada através de uma rede interna criada automaticamente pelo Docker Compose, permitindo que cada serviço comunique de forma segura e transparente com os restantes componentes da arquitetura.

Adicionalmente, foram desenvolvidos scripts automatizados através de um Makefile com o objetivo de simplificar tarefas recorrentes durante o desenvolvimento e operação do sistema. Estes scripts permitem executar operações como a configuração inicial do ambiente, arranque e encerramento dos serviços, ingestão de dados, limpeza de volumes, execução de testes e download dos modelos de linguagem necessários ao funcionamento da aplicação.

Esta abordagem contribuiu para reduzir o tempo de configuração do projeto, aumentar a portabilidade da solução e garantir uma experiência de utilização consistente entre todos os membros da equipa.

7.2.4. Integração Contínua (CI/CD)

O projeto adotou práticas de Continuous Integration (CI) com o objetivo de garantir a qualidade e consistência do código desenvolvido ao longo do semestre.

A integração contínua foi implementada através do GitHub Actions e encontra-se associada ao processo de Pull Request. Sempre que uma nova alteração é submetida para revisão, é automaticamente desencadeado um conjunto de verificações que validam a qualidade do código antes da sua integração na branch principal.

Entre estas verificações destacam-se a análise estática de código, validação de estilo e execução de regras de linting através da ferramenta Ruff. Estas validações permitem identificar problemas de formatação, potenciais erros de programação, código redundante e desvios relativamente às convenções adotadas pela equipa.

A utilização deste processo automatizado permitiu reduzir significativamente a probabilidade de introdução de erros no repositório principal, promovendo simultaneamente boas práticas de desenvolvimento e aumentando a qualidade global do software produzido.

Embora o projeto não implemente um processo completo de Continuous Deployment (CD), a automatização das verificações de qualidade constitui um passo importante na adoção de práticas DevOps modernas, facilitando a manutenção, evolução e escalabilidade futura da plataforma.

A combinação entre GitHub, Pull Requests e pipelines automáticas de validação permitiu estabelecer um fluxo de desenvolvimento mais seguro e controlado, assegurando que todas as contribuições respeitam os mesmos padrões de qualidade antes de serem integradas no sistema.

7.3. Planeamento Semanal

O trabalho da equipa foi organizado de forma iterativa, com acompanhamento contínuo do progresso e redefinição semanal de prioridades. Para a gestão das tarefas foi utilizada a plataforma Jira, onde eram criados tickets associados ao desenvolvimento de novas funcionalidades, correção de bugs, realização de testes, melhorias da interface e integração entre os diferentes componentes do sistema. O *backlog* era atualizado semanalmente de acordo com o estado atual do projeto e os objetivos definidos pela equipa.

A distribuição das tarefas foi realizada com base nas necessidades de cada *sprint* semanal e também nas áreas de especialização dos diferentes membros da equipa. Ao longo do desenvolvimento foram executadas tarefas relacionadas com a implementação do frontend do chatbot, integração entre o chatbot e as dashboards interativas, desenvolvimento das tools utilizadas pelo agente inteligente, testes às respostas geradas pelo modelo de linguagem e processos de ingestão e transformação de dados.

Semanalmente, a equipa realizava apresentações ao professor com o objetivo de demonstrar o progresso alcançado, apresentar funcionalidades já implementadas, discutir impedimentos encontrados e validar o planeamento das tarefas seguintes. Estas reuniões permitiram obter feedback contínuo ao longo do projeto e ajustar prioridades de forma iterativa, contribuindo para uma evolução mais consistente e alinhada com os objetivos definidos.

8. Implementação

Esta secção descreve a implementação técnica dos principais componentes do sistema.

8.1. Pipeline de Indexação

A construção da base de conhecimento do sistema envolveu múltiplas fontes de dados heterogéneas, distribuídas pelas três bases de dados que compõem a infraestrutura de persistência.

Para o armazenamento vectorial no ChromaDB foram criadas duas colecções. A colecção `pmc_medicine_preventive` agrega artigos científicos recolhidos da PubMed Central através da API Entrez da NCBI, utilizando termos MeSH como “preventive medicine”, “disease prevention”, “public health”, “mass screening” e “health promotion”. O crawler efectua pedidos XML à API, extrai o título, o resumo e os parágrafos do corpo do artigo, e filtra os documentos por presença de palavras-chave médicas e por comprimento mínimo de texto. Os documentos aceites são segmentados em fragmentos de 800 caracteres com uma sobreposição de 200 caracteres entre fragmentos consecutivos. Esta configuração preserva o contexto local de cada passagem e evita que frases relevantes fiquem cortadas nos limites dos fragmentos. A colecção `home_remedies` é alimentada a partir de um documento PDF alojado na Universidade de Columbia, transferido directamente por HTTP, processado pela biblioteca `pypdf` para extracção de texto, e segmentado em fragmentos de 500 caracteres com sobreposição de 50. Em ambas as colecções, os fragmentos são convertidos em vectores pelo modelo BAAI/bge-base-en-v1.5 com normalização activada, garantindo que a similaridade co-sinusoidal reflecte correctamente a proximidade semântica independentemente do comprimento do texto. Cada fragmento é armazenado juntamente com metadados de origem como URL, título e palavras-chave associadas à consulta de recolha.

Para a base de dados MongoDB foram ingeridas duas fontes externas via API. O serviço da OMS Global Health Observatory disponibiliza uma API REST a partir da qual são extraídos todos os indicadores de saúde global, as dimensões e os respectivos valores, que ficam guardados nas colecções `gho_indicators`, `gho_dimensions` e em colecções dinâmicas por dimensão, seguindo a convenção de nomenclatura `gho[código]dimension_values`, por exemplo `gho_country_dimension_values` ou `gho_agegroup_dimension_values`. A API MedlinePlus, da Biblioteca Nacional de Medicina dos EUA, é consultada para cada doença registada na base de dados relacional, devolvendo resumos clínicos em formato XML que são processados e armazenados na colecção `medlineplus_health_topics`.

A base de dados relacional PostgreSQL é populada por seis processos ETL independentes. O dataset “Global Health Statistics” em formato CSV fornece estatísticas epidemiológicas globais por país, ano e doença, incluindo taxas de prevalência, incidência e mortalidade, acesso a cuidados de saúde, custos de tratamento e DALYs, armazenados na tabela `global_health_stats`. O dataset BRFSS 2023 do CDC contém respostas individuais a um inquérito de vigilância comportamental nos EUA, com diagnósticos de diabetes, asma, acidente vascular cerebral, doenças cardíacas e cancro, bem como factores de risco como pressão arterial, colesterol, tabagismo, consumo de álcool e índice de massa corporal, que alimentam a tabela `brfss_responses`. O dataset de medicamentos é carregado para a tabela `drugs_side_effects`, contendo nome do fármaco, condição médica associada, efeitos secundários, classe terapêutica, marcas comerciais e avaliações de utilizadores. O dataset WUENIC, fornecido em formato Excel pela OMS e UNICEF, contém estimativas de cobertura vacinal por país e por vacina ao longo dos anos, e é organizado num modelo dimensional composto pelas tabelas `country_dim`, `vaccine_dim` e `immunization_fact`. O dataset de indicadores de doenças crónicas dos EUA, disponibilizado pelo CDC, é carregado na tabela `chronic_disease_indicators` com indicadores ao nível estadual, intervalos de confiança e estratificações demográficas. Por fim, quatro ficheiros CSV com descrições de doenças, precauções, pesos de severidade de sintomas e mapeamentos doença-sintoma são ingeridos nas tabelas `diseases`, `symptoms`, `disease_symptoms` e `disease_precautions`.

8.2. Pipeline de Consulta

Quando uma mensagem chega ao serviço de chat, o primeiro passo é determinar se se trata de um pedido de alteração ao painel de visualização. Esta detecção é feita por expressões regulares que verificam a presença de palavras de acção combinadas com referências ao painel ou a secções específicas como dados crónicos ou imunização. Caso seja detectado um pedido de painel, o fluxo é encaminhado directamente para a ferramenta de dashboard sem passar pelo motor de regras, pois as validações de domínio rejeitariam estas mensagens por baixa similaridade semântica com conteúdo médico.

Para as restantes mensagens, o motor de regras verifica se a pergunta é demasiado curta, se corresponde a uma FAQ pré-definida ou se está fora do domínio médico. Apenas as mensagens que passam estas verificações avançam para o agente de selecção de ferramentas. Este agente envia a mensagem ao modelo de linguagem local via Ollama com temperatura zero, pedindo-lhe que devolva um objecto JSON com a lista de ferramentas a utilizar, por exemplo `{"tools": ["RAG", "SQL"]}`. A resposta é processada e as ferramentas identificadas são executadas por ordem.

Quando a ferramenta RAG é invocada, o sistema começa por embeber a pergunta com o mesmo modelo BAAI/bge-base-en-v1.5 utilizado na indexação. São recuperados os cinco fragmentos mais próximos por similaridade co-sinusal a partir do ChromaDB. Estes candidatos são depois reclassificados pelo modelo BAAI/bge-reranker-base, um CrossEncoder que avalia cada par pergunta-fragmento de forma conjunta, e os três fragmentos com maior pontuação são seleccionados como contexto. O modelo de linguagem recebe então a pergunta e os três fragmentos e gera uma resposta em linguagem natural com base nesse contexto.

A ferramenta SQL segue uma abordagem em dois passos. Primeiro, é construído um esquema compacto da base de dados PostgreSQL contendo apenas os nomes das tabelas e das colunas, sem tipos nem restrições, para minimizar o número de tokens consumidos. Este esquema, acompanhado de contexto semântico sobre o conteúdo das tabelas e da pergunta do utilizador, é enviado ao modelo de linguagem que gera a consulta SQL correspondente. A consulta gerada é extraída da resposta, que pode estar em bloco de código Markdown ou em texto livre, e é validada para garantir que começa com SELECT e não contém palavras reservadas de modificação como INSERT, UPDATE ou DROP. Após a execução na base de dados, os resultados são enviados de volta ao modelo para que este os apresente em linguagem natural ao utilizador.

A ferramenta MongoDB utiliza um catálogo de colecções descrito num ficheiro YAML para planear a consulta. O modelo de linguagem analisa a pergunta e decide qual a acção a executar, como `search_indicators` para pesquisar indicadores da OMS, `get_dimension_values` para listar valores de uma dimensão específica, ou `search_disease_info` para consultar tópicos de saúde do MedlinePlus. Os resultados são formatados pelo modelo numa resposta legível para o utilizador.

A ferramenta de dashboard não gera uma resposta textual extensiva. Em vez disso, invoca um modelo de linguagem local para extrair filtros estruturados da mensagem em linguagem natural, como país, intervalo de anos, nome de doença, código de vacina ou localização geográfica nos EUA. Estes filtros são validados e tipificados antes de serem devolvidos à interface, que os aplica directamente aos gráficos do painel sem necessidade de recarregar a página completa.

8.3. Integração de Componentes

Todos os serviços da aplicação são orquestrados pelo Docker Compose, que define uma rede interna partilhada por todos os contentores. A interface de utilizador desenvolvida em Streamlit corre na porta 8501 e comunica com a API FastAPI na porta 8500 por HTTP, enviando a mensagem do utilizador e recebendo um objecto JSON com a resposta, a ferramenta utilizada e, quando aplicável, os filtros de painel a aplicar. Esta separação entre interface e lógica de negócio permite que cada componente seja escalado ou substituído de forma independente.

A API FastAPI acede ao PostgreSQL através do SQLAlchemy com o conector psycopg2, ao MongoDB através do PyMongo e ao ChromaDB através do cliente HTTP oficial. As ferramentas de geração de linguagem

natural enviam pedidos HTTP ao servidor Ollama na porta 11434, que aloja os modelos de linguagem localmente e evita dependências de serviços externos pagos. O reranker do pipeline RAG é executado directamente no processo Python sem comunicação de rede, uma vez que o modelo CrossEncoder é carregado em memória pela biblioteca sentence-transformers.

A observabilidade de todas as operações é garantida pelo Langfuse, que regista traces e spans para cada etapa do processamento, desde a recepção da mensagem até à resposta final. Cada ferramenta abre um span dedicado com o seu input e output, o que permite auditar o comportamento do sistema em produção e identificar gargalos de latência ou falhas em etapas específicas do pipeline.

9. Avaliação e Métricas

Esta secção apresenta o processo de avaliação do sistema desenvolvido, bem como as métricas utilizadas para validar o seu funcionamento, robustez e capacidade de encaminhamento correto de pedidos para os diferentes componentes da arquitetura. A avaliação foi desenhada com o objetivo de verificar não apenas a qualidade das respostas produzidas, mas também o comportamento do sistema enquanto arquitetura híbrida baseada em regras, ferramentas determinísticas e modelos de linguagem.

O sistema foi avaliado através de dois tipos principais de testes complementares: uma bateria de testes automatizados executada no arranque da aplicação (através de um ficheiro `eval.py`) e um conjunto de testes funcionais realizados diretamente na interface de utilizador (UI evaluation). Em conjunto, estes testes permitem validar tanto o comportamento técnico interno do sistema como a experiência de utilização final.

9.1. Avaliação Automática (Startup Evaluation)

A avaliação automática é executada sempre que a API é iniciada, através do ficheiro `eval.py`. Este mecanismo tem como objetivo garantir que os principais fluxos do sistema continuam funcionais após alterações no código, funcionando como uma forma simplificada de integração contínua. Cada caso de teste define uma entrada representativa, o componente esperado responsável pela resposta e, em alguns casos, uma validação parcial da resposta produzida.

O conjunto de testes inclui cenários de regras (como segurança, perguntas de identidade e filtragem de perguntas não médicas), bem como cenários que validam o correto encaminhamento para ferramentas específicas, como SQL, MongoDB, RAG e o dashboard analítico.

Um exemplo de teste de regras consiste na deteção de urgência médica. Perante a mensagem “I can’t breathe and I have severe crushing chest pain right now”, o sistema deve ativar o módulo de regras e classificar corretamente a situação como emergência médica, sem recorrer a modelos externos. Este comportamento garante que situações críticas são tratadas com prioridade e consistência determinística.

Outro conjunto de testes de regras valida a capacidade do sistema responder a perguntas de identidade e capacidades, como “Who are you?” e “What do you do?”, assegurando que o sistema responde de forma consistente como assistente médico baseado em inteligência artificial, sem introduzir informação clínica incorreta ou diagnósticos.

Foram também incluídos testes de filtragem de conteúdo não médico, como a pergunta “Who won the Super Bowl last year and what was the final score?”, onde o sistema deve rejeitar a consulta enquanto mantém o foco no domínio clínico. Este tipo de validação garante que o sistema respeita o domínio de aplicação definido e evita respostas fora do contexto médico.

No domínio das ferramentas, o sistema foi avaliado em diferentes camadas. No caso da ferramenta MongoDB, foram testadas perguntas como “Which WHO Global Health Observatory indicators mention tuberculosis or TB?”, onde o sistema deve recorrer ao agente de seleção de ferramentas e executar uma query estruturada sobre dados de saúde global. O mesmo acontece com perguntas como “What is hypertension, what causes it, and what are common symptoms?”, onde o sistema combina recuperação via RAG e consulta a dados estruturados para produzir uma resposta mais completa.

No caso da base de dados SQL, foi validado o correto funcionamento de queries estruturadas, como “Using your SQL health database, how many distinct diseases are recorded, and list three disease names from the diseases table.”. Neste cenário, o sistema deve gerar uma query determinística e devolver resultados consistentes, demonstrando a capacidade de resposta a perguntas factuais e estruturadas.

O sistema de RAG (Retrieval Augmented Generation) foi avaliado através de perguntas mais conceptuais e baseadas em literatura científica, como “Explain the pathophysiological mechanisms of insulin resistance and why physical activity improves glycemic control, based on scientific literature.”. Nestes casos, o sistema deve recuperar documentos relevantes, aplicar reranking e gerar uma resposta fundamentada em conhecimento externo indexado.

Por fim, foi avaliado o módulo de dashboard, através de pedidos como “Please show the dashboard and filter chronic disease data for Texas.”. Este tipo de teste valida a capacidade do sistema interpretar intenções de visualização e extrair filtros estruturados para atualização do estado da interface.

O resultado global desta avaliação automática demonstrou um desempenho consistente, com todos os testes a passarem com sucesso, evidenciando a estabilidade do sistema e a correta integração entre os diferentes módulos (ver resultados abaixo).

=== DrHouseGPT startup evaluation (9 cases) ===
(OLLAMA_HOST=http://ollama:11434)

--- RULE-BASED CASES (no LLM routing) ---

Case: rules_emergency

Q: I can't breathe and I have severe crushing chest pain right now.

→ Tool: rules

→ Result: emergency classification triggered

→ Time: 0.1s

[ok]

Case: rules_identity

Q: Who are you?

→ Tool: rules

→ Result: identity FAQ matched (DrHouseGPT assistant description)

→ Time: 0.0s

[ok]

Case: rules_capabilities

Q: What do you do?

→ Tool: rules

→ Result: capability FAQ matched (medical assistant + disclaimer)

→ Time: 0.0s

[ok]

Case: rules_non_medical

Q: Who won the Super Bowl last year and what was the final score?

→ Tool: rules (non-medical rejection)

→ Result: query rejected as non-medical

→ Time: 0.0s

[ok]

--- TOOL ROUTING CASES (LLM / hybrid pipeline) ---

Case: tool_mongo_who

Q: Which WHO Global Health Observatory indicators or indicator codes mention tuberculosis or TB?

→ Tool selection log:

Medical score low → Non-medical score borderline → LLM routing activated

→ Selected tool: mongo_query

→ Execution: MongoDB WHO indicators search

→ Result: structured indicator retrieval

→ Time: 19.0s

[ok]

Case: tool_mongo_disease

Q: What is hypertension, what causes it, and what are common symptoms?

→ Tool selection log:

Medical score: 0.76 | Non-medical score: 0.12 → LLM routing activated

→ Selected tools: rag_answer + mongo_query

→ Execution: RAG retrieval (6 docs) + MongoDB disease enrichment

→ Result: hybrid medical explanation

→ Time: 73.8s

[ok]

Case: tool_sql_stats

Q: Using your SQL health database, how many distinct diseases are recorded, and list three disease names from the diseases table.

→ Tool selection log:

SQL structured intent detected → SQL router triggered

→ Selected tool: sql_query

→ Execution: deterministic SQL aggregation + sampling

→ Result: count + disease list returned

→ Time: 16.7s

[ok]

Case: tool_rag_chroma

Q: Explain the pathophysiological mechanisms of insulin resistance and why physical activity improves glycemic control, based on scientific literature.

→ Tool selection log:

High medical relevance → semantic retrieval required → RAG selected

→ Selected tool: rag_answer

→ Execution: embedding search (BGE) + reranker + LLM synthesis

→ Result: literature-based explanation

→ Time: 26.3s

[ok]

Case: tool_dashboard

Q: Please show the dashboard and filter chronic disease data for Texas.

→ Tool selection log:

Keyword trigger: "dashboard" + filter extraction detected

→ Selected tool: dashboard_query

→ Extracted filter: chronic_location = Texas

→ Result: dashboard state updated with filtered view

→ Time: 8.2s

[ok]

=== FINAL SUMMARY ===

9/9 cases passed

0 failed

0 skipped

9.2. Avaliação Funcional na Interface de Utilizador (UI Evaluation)

Para além dos testes automatizados, foi realizada uma avaliação qualitativa através da interface de utilizador, simulando interações reais com o sistema. Este tipo de avaliação permite validar não apenas o encaminhamento técnico das perguntas, mas também a qualidade das respostas produzidas em contexto de utilização prática.

Um dos cenários testados corresponde à medicina preventiva, um dos principais objetivos do sistema. Foram colocadas questões como “How can I prevent heart diseases?”, “What is a home remedy to treat cough?” e “How does physical activity reduce cardiovascular risk?” (ver [Figura 6](#)). Nestes casos, o sistema recorre maioritariamente ao módulo de recuperação semântica (RAG), complementado com conhecimento estruturado, para fornecer recomendações de prevenção, explicações sobre mecanismos fisiológicos e sugestões de cuidados gerais. Este conjunto de testes é particularmente relevante, uma vez que valida a capacidade do sistema fornecer orientação prática e preventiva em linguagem acessível.

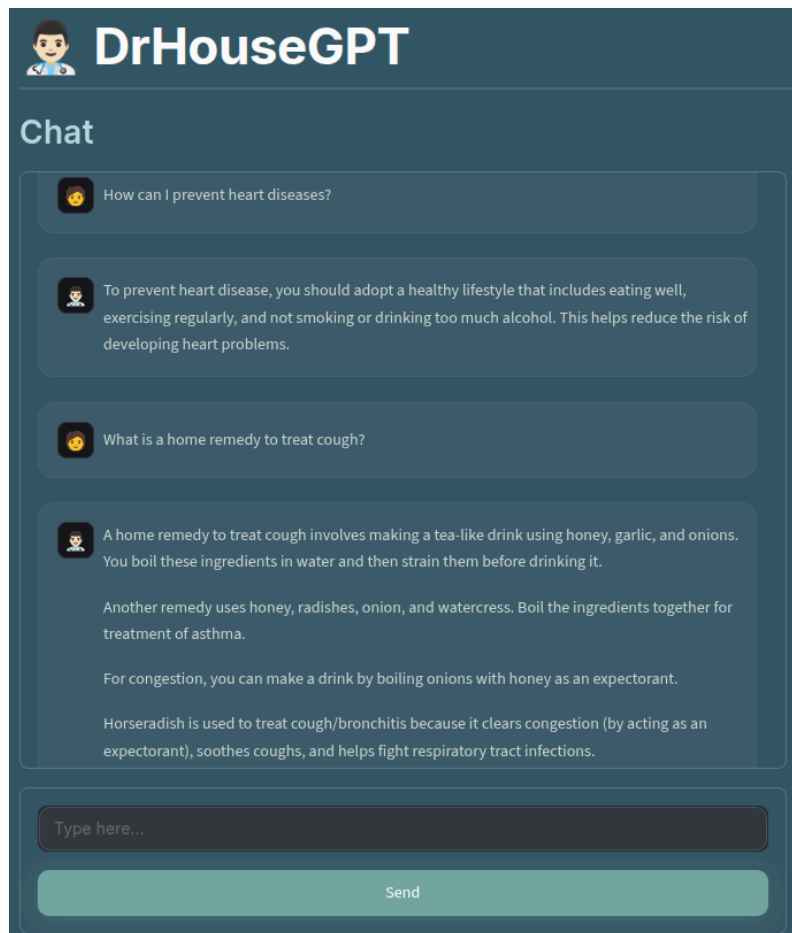


Figura 6: Perguntas de Medicina Preventiva

Outro conjunto de testes focou-se na interpretação de sintomas e avaliação de urgência clínica. Foram simuladas situações como “I fell down the stairs and my head is bleeding everywhere, what should I do?” e “I have been coughing and feeling cold today, should I go to the hospital?”. No primeiro caso, o sistema deve ativar imediatamente o módulo de regras de urgência, classificando a situação como emergência médica e recomendando assistência imediata, podendo ainda dar conselhos de como aliviar a situação até a ajuda profissional chegar. No segundo caso, o sistema deve fornecer uma resposta mais moderada, sugerindo medidas de cuidado pessoal e indicando sinais de alerta que justificam avaliação médica caso os sintomas se agravem (ver [Figura 7](#)).

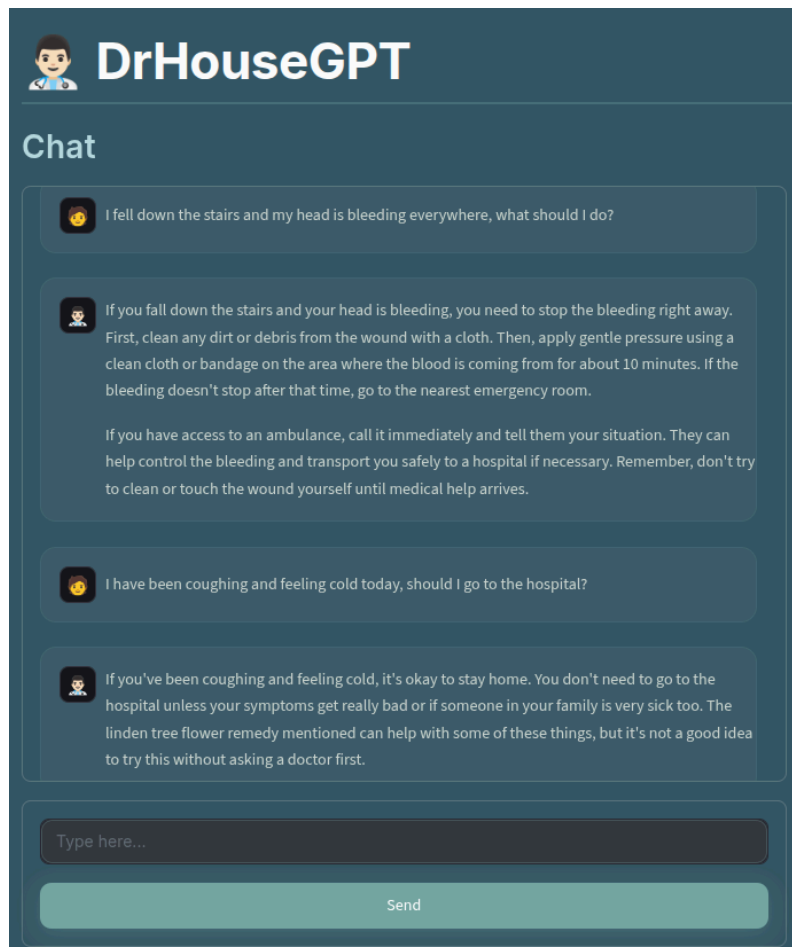


Figura 7: Perguntas de Emergência Médica e sem Emergência Médica

Foram também testadas perguntas de natureza informativa e de identidade, como “Who are you?” e “What do you do?”. Nestes casos, o sistema deve responder de forma consistente, apresentando-se como assistente médico baseado em inteligência artificial, com foco em medicina preventiva e suporte à decisão, reforçando simultaneamente que não substitui aconselhamento médico profissional.

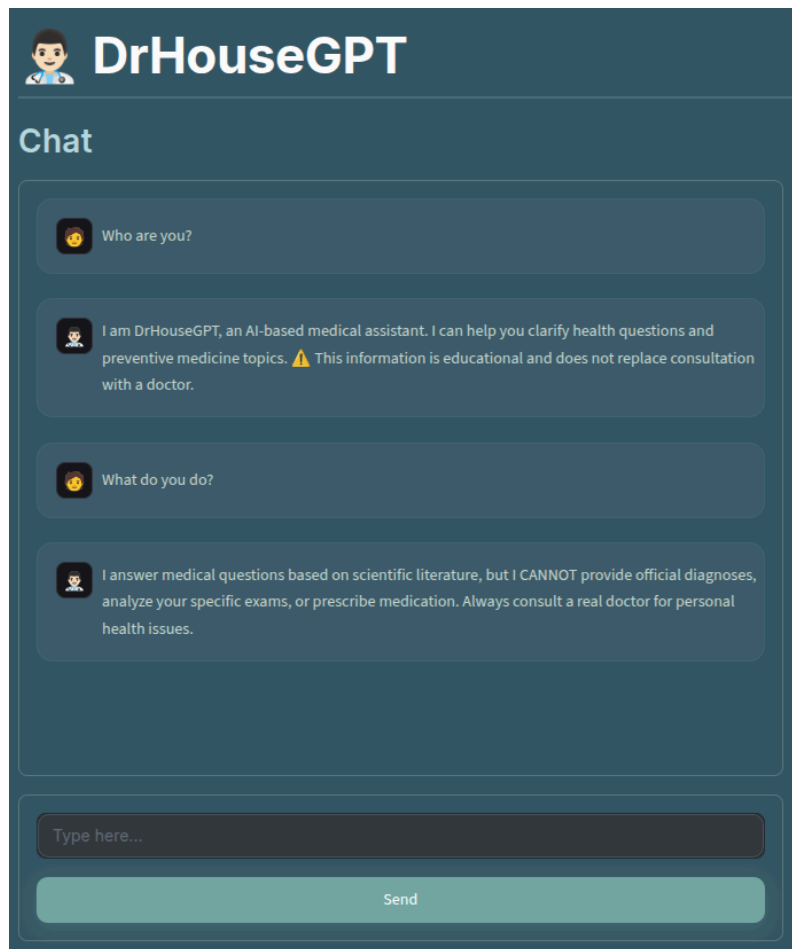


Figura 8: Perguntas de Identidade do Sistema

Por fim, foram incluídas perguntas de conhecimento geral não médico, como “Who won the Super Bowl last year and what was the final score?”. Estes testes são importantes para validar o mecanismo de filtragem de domínio, garantindo que o sistema mantém o foco em questões de saúde e evita respostas fora do âmbito definido (ver [Figura 9](#)).

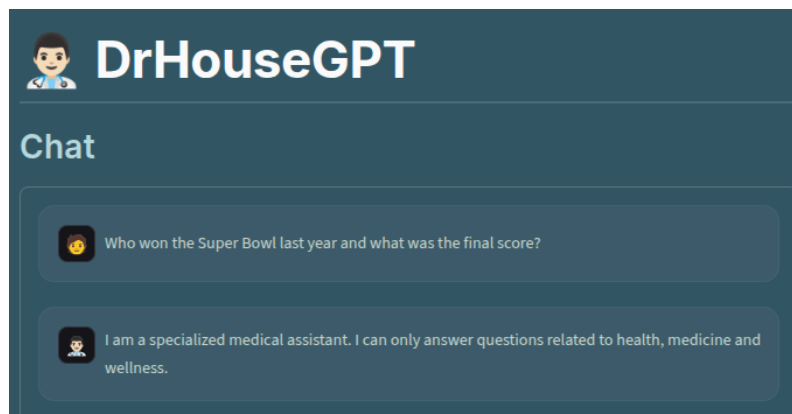


Figura 9: Pergunta Fora de Contexto

A avaliação funcional confirma que o sistema não só encaminha corretamente os pedidos para os módulos adequados, como também produz respostas coerentes, contextualizadas e alinhadas com o objetivo de suporte à decisão em medicina preventiva.

9.3. Síntese da Avaliação

A combinação entre a avaliação automática implementada no `eval.py` e os testes realizados diretamente na interface permite uma validação abrangente do sistema, cobrindo tanto a qualidade dos fluxos internos como a qualidade da experiência de utilização final.

Os testes automáticos asseguram a fiabilidade da arquitetura de routing e a correta integração entre os diferentes componentes do sistema, incluindo regras determinísticas, mecanismos de seleção de ferramentas e módulos baseados em RAG, SQL e MongoDB.

Por outro lado, os testes realizados na interface complementam esta validação ao reproduzirem cenários reais de utilização, permitindo observar o comportamento do sistema em contexto próximo ao do utilizador final.

Em conjunto, estes testes permitiram validar diretamente os casos de uso definidos na [Secção 2](#), que constituíam um dos principais objetivos desta fase de avaliação. Os resultados obtidos demonstram que o sistema responde de forma consistente às diferentes tipologias de perguntas previstas, mantendo coerência funcional entre a especificação conceptual e a implementação prática.

No global, os resultados evidenciam uma arquitetura robusta, modular e corretamente integrada, capaz de combinar regras determinísticas, recuperação de informação estruturada e modelos de linguagem de forma eficaz.

10. ESG e IA Responsável

O desenvolvimento de sistemas de inteligência artificial aplicados à área da saúde levanta desafios relacionados com ética, transparência, segurança e utilização responsável da tecnologia. Neste projeto, foram consideradas várias preocupações associadas ao impacto do sistema tanto para os utilizadores como para a fiabilidade das respostas geradas.

10.1. Ética e Segurança

O *chatbot* desenvolvido tem como objetivo fornecer informação educativa e de apoio relacionada com saúde e medicina preventiva, não substituindo aconselhamento médico profissional. Para garantir uma utilização mais segura do sistema, foram implementadas regras determinísticas para limitar respostas em cenários considerados críticos ou potencialmente perigosos.

Sempre que são identificadas situações relacionadas com emergências médicas, risco para o utilizador ou questões sensíveis, o sistema encaminha o utilizador para serviços apropriados, como contacto com profissionais de saúde ou serviços de emergência. Além disso, o *chatbot* apresenta avisos explícitos indicando que a informação fornecida não substitui uma consulta médica.

Durante o desenvolvimento também foram realizados testes contínuos às respostas geradas pelo modelo de linguagem com o objetivo de identificar limitações, incoerências e possíveis alucinações. A combinação entre ferramentas determinísticas e modelos de linguagem permitiu reduzir parcialmente estes problemas, aumentando a fiabilidade geral do sistema.

10.2. Governança e Transparência

Uma das preocupações do projeto foi aumentar a transparência da interação entre o utilizador e o sistema inteligente. Para esse efeito, foram implementados mecanismos visuais no frontend capazes de informar o utilizador sobre o estado do processamento da pergunta, incluindo indicações como análise da pergunta, consulta de bases de dados, recuperação de documentos e geração da resposta final.

O sistema inclui ainda mecanismos de observabilidade e tracing através da integração do Langfuse, permitindo monitorizar: ferramentas utilizadas pelo agente, tempos de resposta, fluxo de execução e erros ocorridos durante o processamento.

Esta abordagem facilita o *debugging*, auditoria e melhoria contínua do sistema, além de aumentar a capacidade de compreender como determinadas respostas foram produzidas.

10.3. Sustentabilidade e Eficiência

O projeto privilegia a execução local dos modelos de linguagem através do Ollama, reduzindo dependências externas e evitando a necessidade de utilização contínua de serviços cloud de elevado consumo computacional.

Além disso, foram adotadas práticas de automatização e otimização dos pipelines de ingestão de dados e execução do sistema, incluindo utilização de Docker Compose e Makefile. Estas abordagens permitem reduzir tarefas redundantes, melhorar a eficiência operacional e simplificar a manutenção do projeto.

A utilização de modelos de menor dimensão também contribui para uma utilização mais eficiente dos recursos disponíveis, permitindo executar o sistema em ambientes computacionais mais modestos.

11. Conclusão e Trabalho Futuro

11.1. Conclusão

O presente projeto permitiu desenvolver um sistema inteligente de suporte à decisão na área da medicina preventiva, combinando capacidades conversacionais, análise de dados e recuperação de informação médica proveniente de múltiplas fontes heterogêneas. A solução implementada integrou modelos de linguagem, mecanismos de *Retrieval Augmented Generation* (RAG), bases de dados relacionais e documentais, bem como dashboards interativos capazes de serem atualizados através de linguagem natural.

Ao longo do desenvolvimento foram construídos *pipelines* de ingestão e transformação de dados, integradas diferentes bases de conhecimento médicas e desenvolvidas ferramentas especializadas para responder a diferentes tipos de questões. O sistema final demonstrou capacidade para responder a perguntas relacionadas com doenças, indicadores epidemiológicos, medicamentos, sintomas e métricas globais de saúde, combinando abordagens neurais e simbólicas numa arquitetura neuro-simbólica.

Apesar dos resultados alcançados, foram também identificadas algumas limitações, nomeadamente *hallucinations* associadas aos modelos de linguagem locais, dificuldades na geração automática de *queries* SQL complexas e desafios relacionados com seleção automática de ferramentas. Ainda assim, o projeto demonstrou o potencial da integração entre inteligência artificial, engenharia de dados e sistemas analíticos para apoiar processos de decisão em contextos de saúde.

11.2. Trabalho Futuro

Como trabalho futuro, poderão ser exploradas melhorias ao nível da robustez e escalabilidade do sistema. Uma das principais evoluções passa pela utilização de modelos de linguagem mais avançados e especializados para tarefas médicas, permitindo reduzir erros de *routing* e melhorar a qualidade das respostas produzidas.

Outra possível melhoria consiste na evolução da SQL Tool através da utilização de agentes mais sofisticados, mecanismos de *self-correction* e *frameworks* especializadas em geração de *queries* SQL. Poderão também ser adicionadas estratégias híbridas de *routing* entre regras determinísticas e seleção automática baseada em LLMs.

Por fim, seria relevante realizar avaliações formais da qualidade das respostas do sistema com utilizadores reais ou especialistas da área da saúde, permitindo medir a utilidade prática da solução em cenários de suporte à decisão.

Anexo A. ADRs — Decisões de Engenharia

Durante a construção de um sistema de software complexo, diversas escolhas tecnológicas e arquiteturais precisam de ser realizadas, envolvendo frequentemente diferentes alternativas possíveis, cada uma com vantagens e limitações próprias. A documentação destas decisões permite compreender melhor a evolução do sistema e as razões que justificam determinadas opções técnicas.

Assim, esta secção apresenta as principais decisões de arquitetura tomadas ao longo do desenvolvimento do sistema.

Para garantir maior transparência e rastreabilidade das escolhas realizadas, foi adotada uma abordagem baseada em **Architecture Decision Records (ADR)**. Esta metodologia permite registar de forma estruturada as decisões relevantes de arquitetura, incluindo o contexto em que surgem, a solução adotada e as alternativas consideradas.

A utilização de ADRs facilita também a manutenção e evolução futura do sistema, uma vez que fornece um registo claro das motivações que levaram à seleção de determinadas tecnologias ou padrões arquiteturais.

A.A. ADR-001 — Escolha da Base de Dados Vetorial

Contexto: O sistema requer pesquisa semântica sobre documentos biomédicos, suportando consultas por similaridade de embeddings. A base de dados deve ser compatível com Python, fácil de integrar e suportar armazenamento local de vetores.

Decisão: Selecionou-se **ChromaDB** como base de dados vetorial.

Racional:

- Integração nativa com Python e pipelines de embeddings.
- Suporte a embeddings persistentes e consultas de similaridade.
- Desdobramento simples via Docker, alinhando com a arquitetura do projeto.

Alternativas Consideradas:

- FAISS
- Weaviate
- Pinecone

Consequências:

1. Integração rápida e consistente com pipeline de embeddings.
2. Permite deploy local sem dependências externas.
3. Escalabilidade horizontal limitada em comparação com soluções cloud-native.

A.B. ADR-002 - Escolha da base de dados estruturados

Contexto: O sistema desenvolvido necessitava de uma base de dados relacional capaz de armazenar grandes volumes de dados estruturados relacionados com indicadores epidemiológicos, métricas globais de saúde, vacinação, doenças crónicas, medicamentos e fatores de risco. Era necessário suportar *queries* analíticas, filtros complexos, agregações e relações entre diferentes entidades médicas.

Decisão: Foi decidido utilizar PostgreSQL como base de dados principal para armazenamento de dados estruturados do sistema.

A escolha do PostgreSQL deveu-se à sua robustez, maturidade, suporte para *queries* analíticas complexas e boa integração com bibliotecas Python utilizadas no *backend* do projeto. A utilização de SQL permitiu também criar ferramentas de consulta em linguagem natural através de geração automática de *queries* SQL com recurso a modelos de linguagem.

Racional: A utilização de PostgreSQL permitiu garantir uma estrutura consistente para os dados e facilitar operações analíticas envolvendo agregações, filtros temporais e comparações entre diferentes indicadores de saúde. Além disso, a natureza relacional dos *datasets* utilizados adequava-se naturalmente ao modelo relacional disponibilizado pelo PostgreSQL. A equipa considerou ainda a facilidade de integração com Docker e ferramentas utilizadas no pipeline de desenvolvimento.

Alternativas consideradas:

- MySQL
- SQLite
- DuckDB

Consequências: A adoção de PostgreSQL trouxe vantagens significativas ao projeto, nomeadamente a possibilidade de executar *queries* complexas de forma eficiente e organizar os dados de forma estruturada e consistente. Esta decisão facilitou o desenvolvimento da SQL Tool utilizada pelo agente inteligente e permitiu integrar múltiplos datasets relacionais numa única plataforma analítica. Por outro lado, verificaram-se algumas dificuldades associadas à geração automática de *queries* SQL por modelos de linguagem pequenos, exigindo mecanismos adicionais de validação, simplificação de schema e routing híbrido para aumentar a fiabilidade das respostas produzidas pelo sistema.

A.C. ADR-002 - Escolha da base de dados semi-estruturados

Contexto: O sistema necessitava de armazenar documentos médicos semi-estruturados provenientes de diferentes fontes, incluindo descrições de doenças, informação da MedlinePlus e indicadores da WHO Global Health Observatory. Os dados apresentavam estruturas heterogêneas e variáveis, dificultando a sua modelação num esquema relacional rígido.

Decisão: Foi decidido utilizar MongoDB para armazenamento de dados semi-estruturados e documentos textuais utilizados pelo sistema.

MongoDB foi integrado no *backend* para suportar consultas flexíveis sobre documentos médicos e permitir ao agente inteligente recuperar rapidamente informação descritiva sobre doenças, sintomas e indicadores de saúde.

Racional: A escolha do MongoDB foi motivada pela flexibilidade do modelo documental, que permitia armazenar dados heterogêneos sem necessidade de schemas rígidos. Esta abordagem revelou-se adequada para conteúdos médicos provenientes de múltiplas fontes externas, onde os campos disponíveis variavam entre documentos. Além disso, MongoDB apresentou integração simples com Python e com os serviços backend implementados pela equipa.

Consequências: A utilização de MongoDB permitiu simplificar o armazenamento e recuperação de documentos médicos semi-estruturados, reduzindo a complexidade associada à modelação relacional destes dados. A flexibilidade documental facilitou a ingestão de novas fontes de informação e acelerou o desenvolvimento da componente de pesquisa textual do sistema. No entanto, esta abordagem também introduziu desafios relacionados com normalização de dados e consistência entre documentos provenientes de diferentes *datasets*, exigindo validações adicionais durante os processos de ingestão e transformação de dados.

Alternativas Consideradas:

- PostgreSQL JSONB;
- Elasticsearch;

A.D. ADR-004 — Escolha do Modelo de Embeddings

Contexto: Para indexar os documentos biomédicos, é necessário transformar texto em vetores semânticos. O modelo deve ser robusto, pré-treinado em domínio geral/biomédico, e integrável com Python.

Decisão: Selecionou-se o modelo **BAAI/bge-base-en-v1.5** para gerar embeddings.

Racional:

- Modelo leve e rápido para inferência local.
- Produz embeddings de alta qualidade para tarefas de similaridade semântica.
- Compatível com pipeline Python + ChromaDB.

Alternativas Consideradas:

- Sentence-BERT — boa qualidade, mas maior latência em alguns datasets grandes.
- OpenAI embeddings — cloud-based, custos e dependência externa.

Consequências:

1. Boa relação entre desempenho e velocidade de indexação.
2. Permite manter o workflow 100% local.
3. Pode não capturar nuances biomédicas tão específicas quanto modelos especializados no domínio.

A.E. ADR-005 — Escolha do Modelo LLM

Contexto: Para responder perguntas ou gerar texto a partir do corpus biomédico, o sistema precisa de um modelo de linguagem grande (LLM) com bom equilíbrio entre qualidade e custo computacional.

Decisão: Selecionou-se **Gemma3:4B** como LLM principal.

Racional:

- Modelo 4B adequado para inferência local em hardware disponível.
- Bom desempenho em compreensão de texto biomédico e geração de respostas coerentes.
- Compatível com o fluxo RAG (Retriever-Augmented Generation).

Alternativas Consideradas:

- GPT-3 / GPT-4 — excelente qualidade, mas cloud-based e com custos elevados.
- MPT / LLaMA — podem ser usados localmente, mas menor performance na geração de respostas em inglês biomédico.

Consequências:

1. Permite integração local no pipeline de RAG.
 2. Balanceia qualidade de resposta e custo computacional.
- Limitações de capacidade e contexto comparadas a modelos maiores (ex.: 13B ou 70B).

A.F. ADR-006 — Escolha da API

Contexto: A aplicação necessita de um componente responsável por gerir a comunicação entre o frontend (Streamlit) e os serviços de backend, incluindo o sistema de Retrieval-Augmented Generation (RAG), as bases de dados e o modelo de linguagem executado através do Ollama. Era necessário escolher uma framework para desenvolver uma API que permitisse expor endpoints para receber pedidos do utilizador, processar as perguntas e devolver respostas geradas pelo modelo. A solução deveria ser leve, fácil de integrar com Python e compatível com a arquitetura baseada em containers Docker utilizada no projeto.

Decisão: Foi escolhida a framework FastAPI para implementar a API do sistema. A API corre num container Docker dedicado e expõe endpoints HTTP, nomeadamente o endpoint /chat, que recebe perguntas do utilizador e executa a pipeline RAG [e ferramentas de SQL] antes de devolver a resposta gerada pelo modelo de linguagem.

Racional: A escolha do FastAPI foi motivada por vários fatores. Em primeiro lugar, trata-se de uma framework moderna e eficiente para desenvolvimento de APIs em Python, oferecendo alto desempenho e suporte nativo para programação assíncrona. Além disso, o FastAPI integra-se facilmente com bibliotecas utilizadas no projeto, como as responsáveis pela geração de embeddings e comunicação com o Ollama. Outro fator importante foi a geração automática de documentação interativa através do Swagger (/docs), que facilita o teste e validação dos endpoints durante o desenvolvimento. Finalmente, a framework adapta-se bem a ambientes containerizados, permitindo que a API funcione de forma isolada e comunique facilmente com os restantes serviços através da rede Docker.

Alternativas Consideradas: Foram consideradas outras abordagens, como o uso de Flask, que é uma framework amplamente utilizada para desenvolvimento de APIs em Python. No entanto, o Flask exige mais configuração manual e não fornece, por defeito, funcionalidades como validação automática de dados ou documentação interativa. Outra alternativa seria integrar diretamente a lógica da aplicação no frontend Streamlit, mas essa abordagem dificultaria a separação de responsabilidades e tornaria o sistema menos modular e menos reutilizável.

Consequências: A utilização do FastAPI permitiu criar uma camada de API clara e modular entre o frontend e os serviços de backend. Isto facilita a manutenção do código, melhora a organização da aplicação e permite que a API seja reutilizada por outros clientes ou serviços no futuro.

A.G. ADR-007 — Adoção da Framework LangChain para Consultas SQL

Contexto: Era necessário uma solução que permitisse à LLM realizar introspeção automática do esquema da base de dados, garantisse a segurança das queries executadas e facilitasse a integração de múltiplos componentes de IA.

Decisão: Foi decidida a adoção da framework LangChain (especificamente os módulos langchain_community e langchain_ollama) para gerir a lógica de interação com a base de dados SQL e a orquestração do agente. A implementação utiliza o objeto SQLDatabase para ligar ao PostgreSQL e ferramentas de geração de SQL que utilizam o esquema dinâmico da base de dados como contexto para o modelo.

Racional: A escolha da LangChain justifica-se por três motivos principais:

- **Introspeção Dinâmica:** Através do SQLAlchemy, a LangChain consegue extrair automaticamente o esquema atualizado da base de dados (tabelas, colunas e tipos), fornecendo-o à LLM sem necessidade de atualizações manuais no código sempre que a base de dados muda.
- **Abstração e Robustez:** A framework oferece utilitários para validação de queries e tratamento de erros (ex: run_no_throw), o que torna a aplicação mais resiliente a falhas de sintaxe geradas pelo modelo.
- **Segurança e Controlo:** A integração permite implementar facilmente camadas de segurança, como a filtragem de palavras-chave proibidas (ex: DROP, DELETE, UPDATE), garantindo que a LLM apenas executa operações de leitura (SELECT).

Ecossistema: Sendo uma ferramenta padrão na indústria, facilita a expansão futura do projeto, como a criação de agentes mais complexos ou a integração de novas fontes de dados.

Alternativas Consideradas: A alternativa principal era manter a orquestração manual iniciada no ficheiro pipeline.py. No entanto, essa abordagem exigia a manutenção manual de um dicionário extenso com a descrição do schema, o que era propenso a erros e tornava o prompt demasiado longo (consumo excessivo de tokens).

Consequências: A utilização da LangChain resultou numa redução significativa da lógica “hardcoded” no backend. O sistema tornou-se mais inteligente, pois a LLM agora compreende a estrutura da base de dados em tempo real.

A.H. ADR-008 — Implementação do Langfuse para Observabilidade e Tracing

Contexto: Com o aumento da complexidade do chatbot e a integração de múltiplas ferramentas (SQL, Mongo, RAG), tornou-se difícil monitorizar o fluxo de execução das perguntas, diagnosticar falhas em etapas específicas da “chain” e avaliar o custo ou latência das chamadas às LLMs. Era necessária uma solução que permitisse visualizar a árvore de execução (traces) e armazenar os logs de forma estruturada para auditoria e melhoria do modelo.

Decisão: Foi decidida a integração da plataforma Langfuse como ferramenta central de observabilidade. A implementação utiliza o langfuse_client para criar spans e traces personalizados em cada ferramenta (ex: mongo_tool, sql_tool), capturando payloads de entrada, planos de execução e respostas finais.

Racional: A escolha do Langfuse justifica-se por quatro motivos principais:

- **Rastreamento Detalhado (Tracing):** Permite decompor uma única pergunta do utilizador em várias etapas, identificando exatamente se um erro ocorreu na seleção da ferramenta, na consulta à base de dados ou na geração da resposta final.
- **Monitorização de Prompt e Contexto:** O sistema regista o contexto enviado à LLM (como os dados extraídos do MongoDB), facilitando a deteção de alucinações causadas por dados de entrada ruidosos ou mal formatados.
- **Debugging de Produção:** Através do decorador de logging e do cliente de telemetria, a equipa consegue analisar falhas em tempo real sem necessidade de aceder diretamente aos logs brutos dos contentores Docker.
- **Facilidade de Integração:** A framework possui SDKs nativos para Python que se integram de forma assíncrona, minimizando o impacto na latência da aplicação.

Alternativas Consideradas: A alternativa principal era a utilização exclusiva de logs de texto tradicionais (via biblioteca logging do Python). No entanto, esta abordagem revelou-se insuficiente para sistemas baseados em agentes, pois não permite correlacionar facilmente chamadas aninhadas nem visualizar de forma gráfica o fluxo de dados entre diferentes bases de dados e o modelo de linguagem.

Consequências: A adoção do Langfuse introduziu uma camada de transparência essencial no ciclo de desenvolvimento. Agora, cada interação é auditável, permitindo à equipa realizar testes de stress mais eficazes e identificar gargalos de performance. Como consequência negativa, existe uma dependência adicional de um serviço externo (ou contentor dedicado) e a necessidade de gerir chaves de API e credenciais de segurança no ambiente de produção.

A.I. ADR-009 — Introdução de Makefile para Automação do Setup e Pipeline

Contexto: Com a evolução do projeto, o processo de configuração do ambiente tornou-se progressivamente mais complexo, envolvendo múltiplos passos manuais: criação e configuração do ficheiro .env, build e arranque dos containers Docker, ingestão de dados em diferentes bases de dados (PostgreSQL, MongoDB, ChromaDB) e instalação de modelos LLM via Ollama. Esta sequência de comandos aumentava a probabilidade de erro humano, dificultava a replicação do ambiente por outros membros da equipa e comprometia a reprodutibilidade do sistema.

Decisão: Foi decidida a introdução de um Makefile como interface unificada para automatizar as tarefas principais do projeto. Foram definidos comandos como make setup, make ingest, make pull-model, make lint e make format, permitindo executar pipelines completos com um único comando e reduzindo a necessidade de intervenção manual.

Racional: A adoção do Makefile justifica-se pelos seguintes fatores:

- **Automatização do Setup:** Permite configurar todo o ambiente (containers, variáveis de ambiente, dados e modelos) com um único comando, reduzindo erros manuais.
- **Reprodutibilidade:** Garante que todos os membros da equipa executam exatamente os mesmos passos, evitando inconsistências entre ambientes locais.
- **Simplicidade de Utilização:** Abstrai a complexidade dos comandos Docker e scripts Python, facilitando o onboarding de novos utilizadores.
- **Organização do Workflow:** Centraliza operações frequentes (logs, ingestão, validação, etc.), tornando o processo de desenvolvimento mais estruturado.

Alternativas Connsideradas: Uma alternativa seria documentar os comandos necessários num ficheiro README e executá-los manualmente. No entanto, esta abordagem mostrou-se propensa a erros e difícil de manter, especialmente com a evolução contínua do projeto. Outra alternativa seria a utilização de pipelines CI/CD completos (ex: GitHub Actions), mas foi considerada excessiva para o contexto atual de desenvolvimento local.

Consequências: A introdução do Makefile simplificou significativamente o processo de desenvolvimento e execução do projeto, tornando o pipeline mais robusto e acessível. Como consequência positiva, reduziu-se o tempo de setup e a probabilidade de erros humanos. Como desvantagem, introduz-se uma camada adicional de abstração que requer manutenção e pode esconder detalhes importantes da execução para utilizadores menos experientes.

A.J. ADR-010 — Framework Streamlit para a Dashboard

Contexto:

O projeto necessitava de uma interface visual capaz de apresentar *dashboards* interativos e permitir integração rápida com o *chatbot* inteligente desenvolvido pela equipa. A solução deveria permitir visualização de dados médicos e epidemiológicos, atualização dinâmica de filtros e desenvolvimento rápido durante o período académico do projeto.

Foram consideradas diferentes abordagens para o *frontend* analítico, incluindo *frameworks* web tradicionais e bibliotecas de visualização de dados.

Decisão:

Foi decidido utilizar o *framework* Streamlit para o desenvolvimento da *dashboard* principal do sistema.

A *dashboard* foi integrada diretamente com o *chatbot*, permitindo que determinadas instruções em linguagem natural atualizassem automaticamente filtros e visualizações apresentadas ao utilizador.

Racional:

Alternativas Connsideradas:

- React + FastAPI;
- Dash Plotly;
- Flask com frontend separado.

Consequências: A utilização de Streamlit permitiu acelerar significativamente o desenvolvimento da interface visual do sistema, facilitando a criação de dashboards interativos diretamente em Python e reduzindo a complexidade associada ao desenvolvimento frontend tradicional. Esta decisão tornou possível integrar rapidamente os gráficos, filtros e componentes do chatbot numa única aplicação, permitindo também implementar funcionalidades de atualização dinâmica da dashboard através de linguagem natural. A simplicidade de integração com bibliotecas de análise de dados e visualização revelou-se particularmente vantajosa no contexto académico do projeto.

A.K. ADR-011 - Mudança para qwen2.5:1.5b

Contexto:

O projeto necessitava de uma interface visual capaz de apresentar *dashboards* interativos e permitir integração rápida com o *chatbot* inteligente desenvolvido pela equipa. A solução deveria permitir visualização de dados médicos e epidemiológicos, atualização dinâmica de filtros e desenvolvimento rápido durante o período académico do projeto.

Foram consideradas diferentes abordagens para o *frontend* analítico, incluindo *frameworks* web tradicionais e bibliotecas de visualização de dados.

Decisão:

Foi decidido utilizar o *framework* Streamlit para o desenvolvimento da *dashboard* principal do sistema.

A *dashboard* foi integrada diretamente com o chatbot, permitindo que determinadas instruções em linguagem natural atualizassem automaticamente filtros e visualizações apresentadas ao utilizador.

Racional:

Alternativas Consideradas:

- gemma3:1b;
- gemma3:4b;
- mistral;
- manutenção do modelo inicial.

Consequências: A mudança para o modelo qwen2.5:1.5b permitiu melhorar a qualidade geral das respostas produzidas pelo sistema, especialmente em tarefas relacionadas com seleção de ferramentas e geração de *queries* SQL. O modelo demonstrou maior capacidade de seguir instruções e produzir outputs mais consistentes relativamente aos modelos anteriormente testados, mantendo simultaneamente compatibilidade com execução local através do Ollama. Esta decisão contribuiu para aumentar a estabilidade do agente inteligente e reduzir alguns erros observados durante os testes iniciais. Apesar destas melhorias, continuaram a existir limitações em perguntas mais ambíguas ou *queries* SQL complexas, tornando necessário implementar mecanismos adicionais de validação, *fallback* e *routing* híbrido para aumentar a robustez do sistema.